Cloud Native Data Engineering

Week 5. Data Warehousing and Lakehouse Architectures

Ashish Pujari | apujari@uchicago.edu

Lecture Outline

1. Modern Data Architecture
2. Data Warehouse Architecture
3. Dimensional Modeling
4. Lakehouse Architecture and Governance



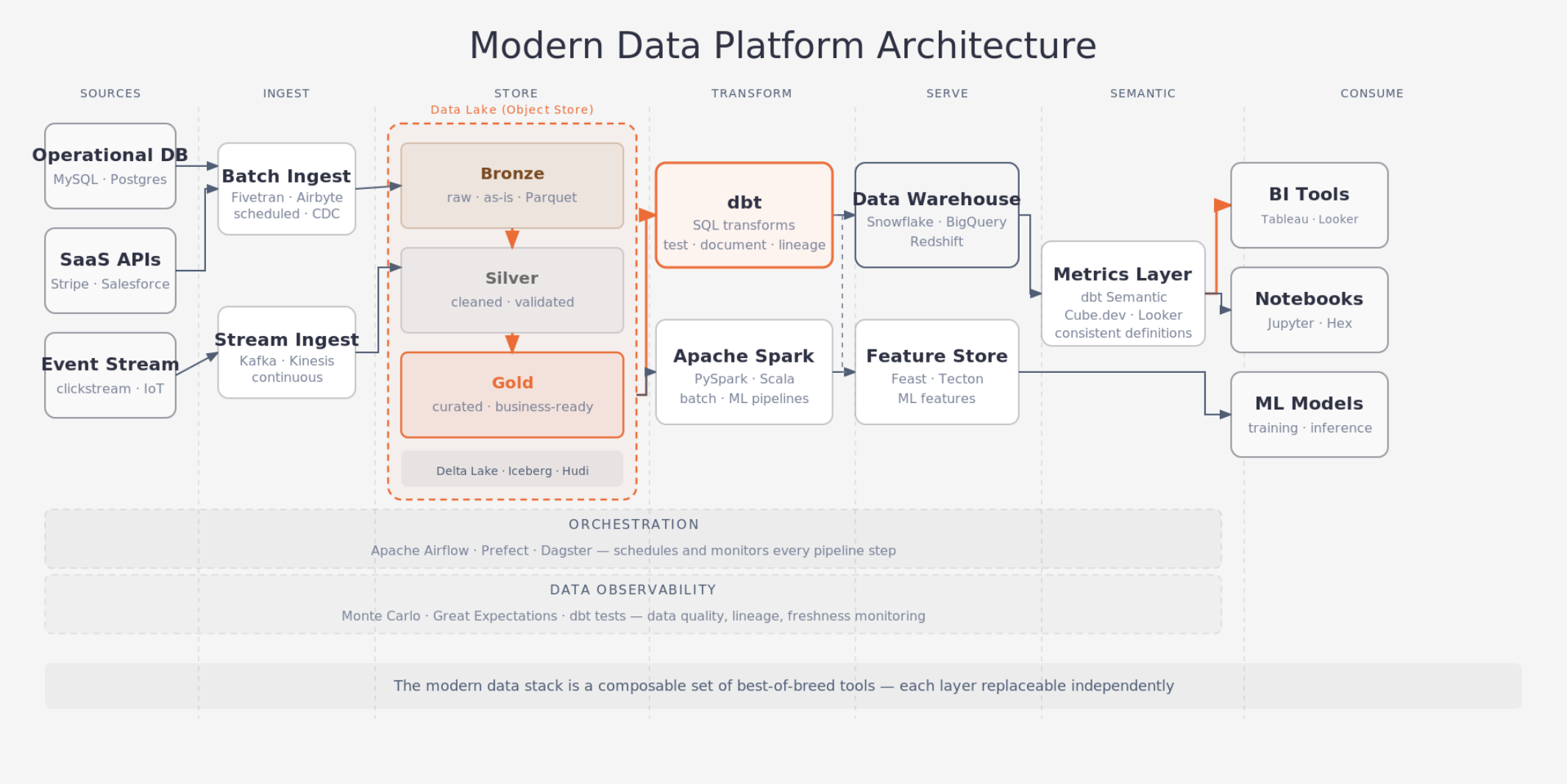
Modern Data Architecture

Module 1

Modern Data Architecture

- Built for end-users and self-service analytics
- Automated with data pipelines and data flows
- Resilient and fault tolerant
- Scalable to meet unpredictable demands
- Enables collaboration and trust
- Provides data governance and security by design

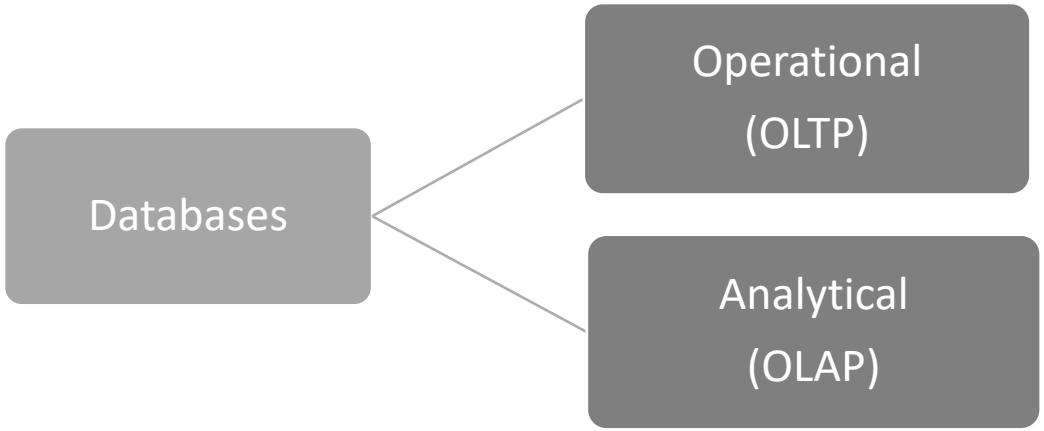
Modern Data Platform Overview

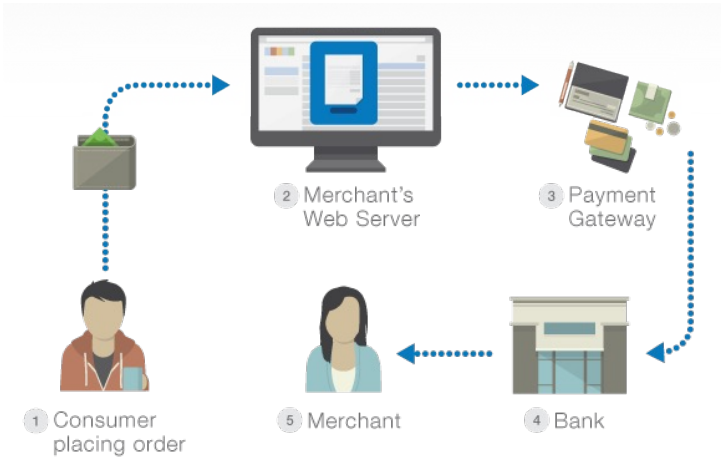


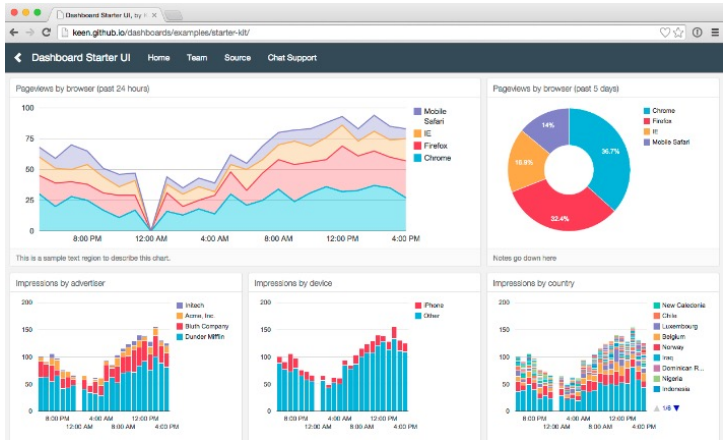
Analytical Databases

- Analytical database
  - Stores and manages business data, market and customer data for business intelligence (BI) analysis
  - Collects historical data related to operations' KPIs and metrics such as sales, performance, and inventory and is easily accessible.
- Why analytical systems
  - Operational tasks vs analytical query performance
  - Impossible to structure a database for both operational and analytics
- Data warehouse: single source of truth
  - Subject-oriented, non-volatile, historical

Database Types - Function







OLTP vs OLAP

	Transactional (OLTP)	Analytical (OLAP)
Application	Online, Transactional	Reporting, Business Intelligence
Operations	Update	Retrieval
Information	Transactional	Actionable
Data Age	Current	Recent and Historical
Data Size	< 100 GB	> 100 GB
Data model	Entity-relationship	Multi-dimensional
Normalization	Normalized	De-normalized
Function	Application Oriented	Subject Oriented

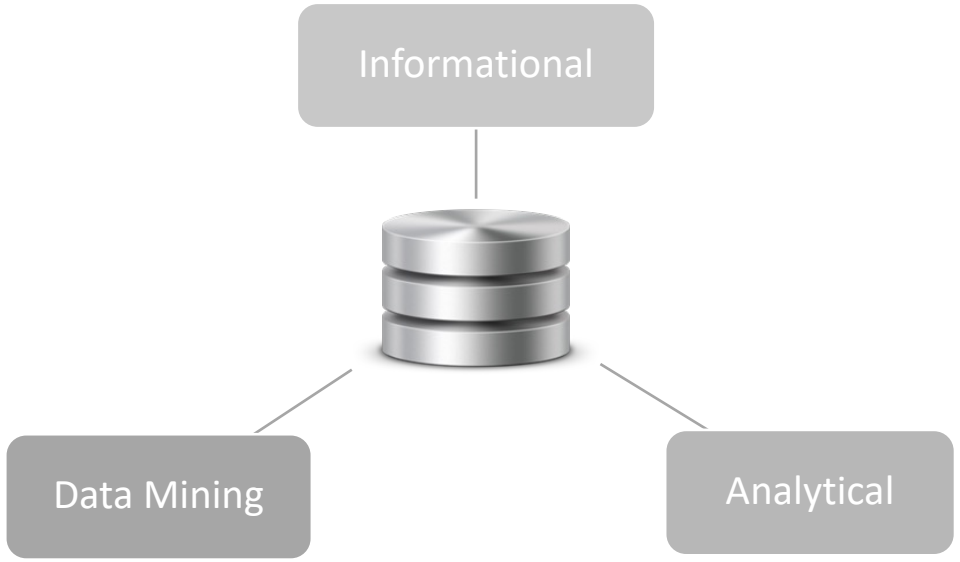


Data Warehouse Architecture

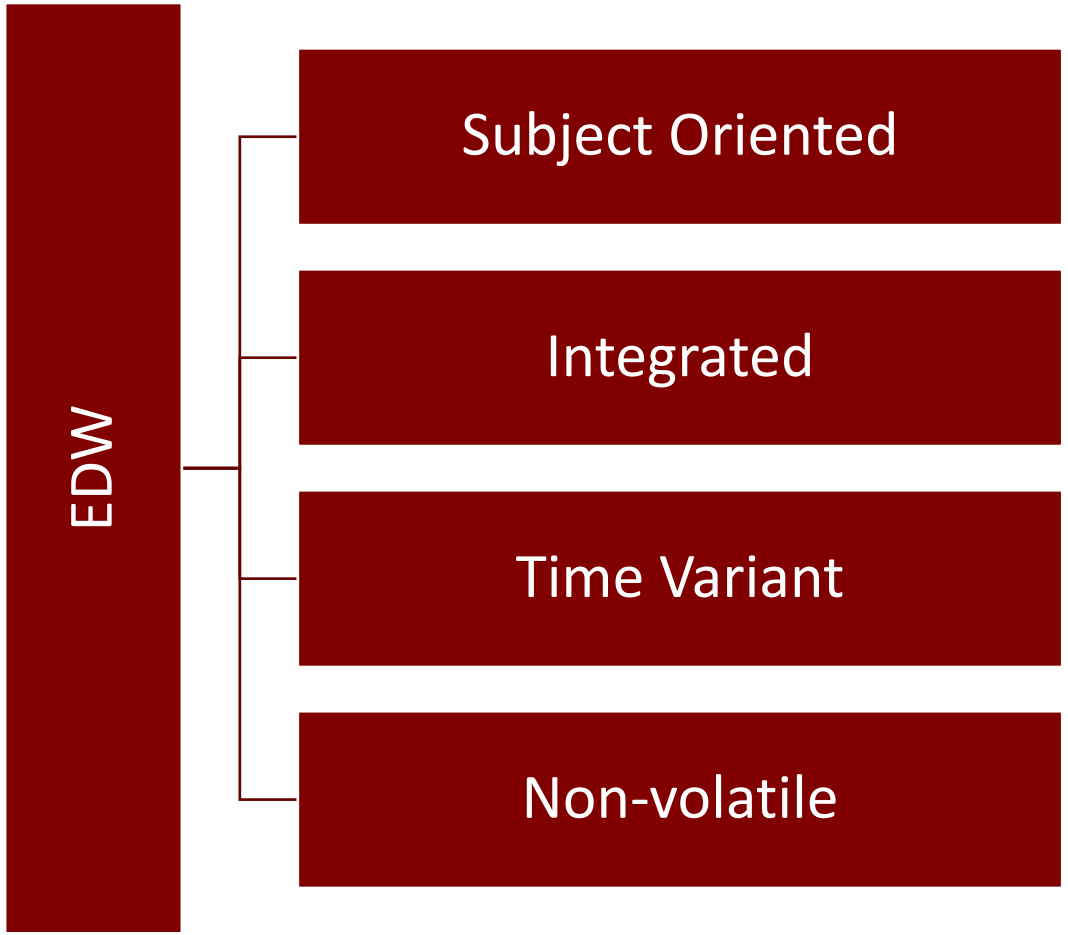
Module 2

Enterprise Data Warehouse (EDW)

- The purpose of the enterprise data warehouse is the retrieval of analytical information. It can store detailed and/or summarized historical data



Enterprise Data Warehouse (EDW)



Developed to analyze specific business subject areas

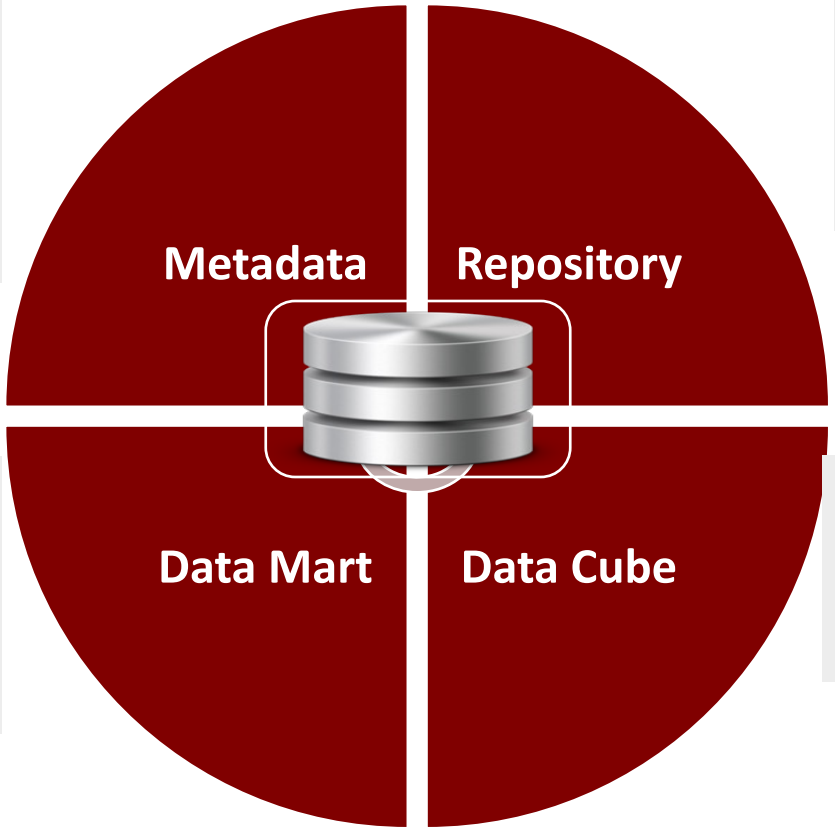
Process of bringing the data from multiple operational data sources into a singular data warehouse

Contains slices or snapshots of data from different periods of time

Once entered into the warehouse, data should not be updated or erased

Data Warehouse Components

- Data elements and attributes of those elements
- Structure of the data (fields, length, columns)
- Helps enterprise systems locate the contents of a DW



- Database of Metadata
- Classification
  - Business Metadata
  - Technical Metadata
  - Summarization rules

- Subset of data from a data warehouse; smaller in size
- Data specific to a single line of business or department e.g., Finance or Marketing

- Multi-dimensional array of values
- Data abstraction to evaluate aggregated data from a variety of viewpoints

Application Oriented - Example

Customers

customerNumber	customerName	contactLastName
103	Atelier graphique	Schmitt
112	Signal Gift Stores	King
114	Australian Collectors, Co.	Ferguson
119	La Rochelle Gifts	Labrune
121	Baane Mini Imports	Bergulfsen

orderNumber	orderDate	requiredDate	shippedDate	status
10100	2016-01-06	2016-01-13	2016-01-10	Shipped
10101	2016-01-09	2016-01-18	2016-01-11	Shipped
10102	2016-01-10	2016-01-18	2016-01-14	Shipped
10103	2016-01-29	2016-02-07	2016-02-02	Shipped

Orders

orderNumber	productCode	quantityOrdered	priceEach	orderLineNumber
10100	S18_1749	30	136.00	3
10100	S18_2248	50	55.09	2
10100	S18_4409	22	75.46	4
10100	S24_3969	49	35.29	1

Orderdetails

Subject Oriented - Example

Total Units ordered on a monthly basis based on country & product Line

Amount spent and units bought per customer/month

contactFirstName	contactLastName	phone	month	units	amountSpent
Anna	O'Hara	02 9936 8555	January	62	9025.1
Adrian	Huxley	+61 2 9495 8555	July	193	21639.47
Anna	O'Hara	02 9936 8555	March	61	9162.79

orderNumber	productCode	customerNumber	employeeNumber	dateId	quantityOrdered	priceEach
10100	S18_1749	363	1216	735	30	136
10100	S18_2248	363	1216	735	50	55.09
10100	S18_4409	363	1216	735	22	75.46
10100	S24_3969	363	1216	735	49	35.29

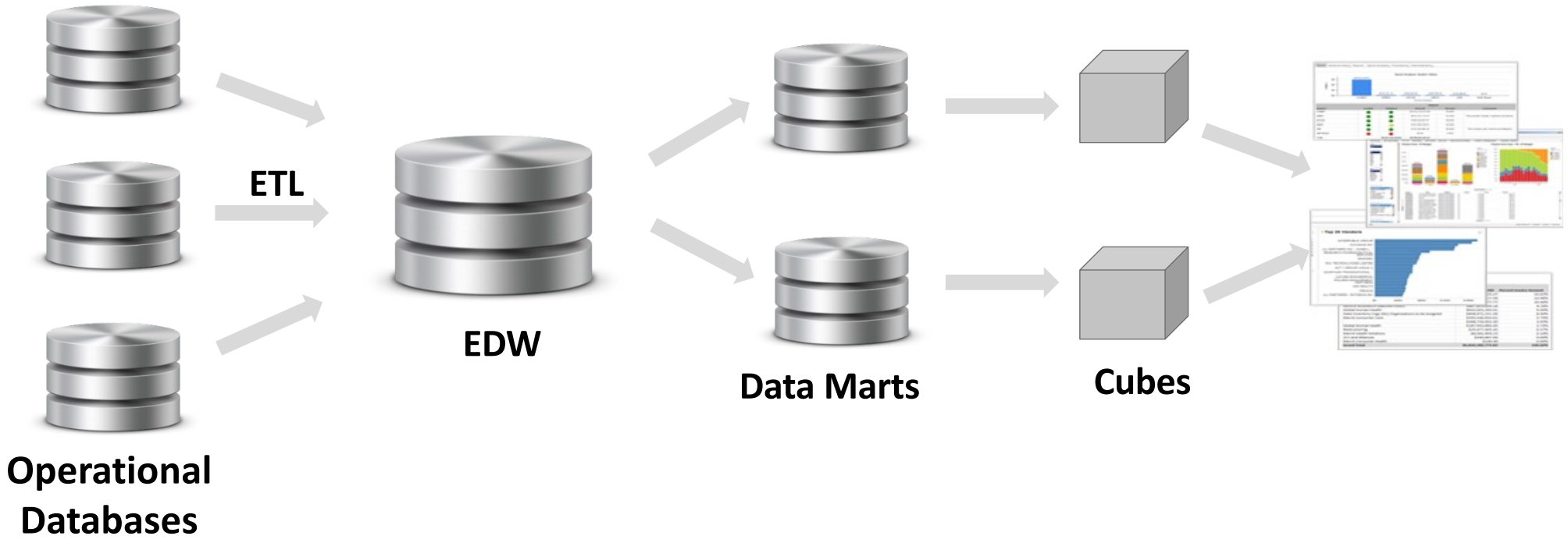
Fact Table for analysis of orders

EDW Data Pipeline



Data Marts and Cubes

- A data mart is a subject-oriented database that is often a partitioned segment of an enterprise data warehouse



Inmon vs Kimball Approach

- Inmon (top-down): build normalized EDW first; derive dimensional data marts from it
- Kimball (bottom-up): build dimensional data marts; integrate via data warehouse bus
- Inmon: more consistent enterprise-wide; longer initial build time
- Kimball: faster time-to-value for specific business questions; risk of mart sprawl
- Modern approach: ELT with dbt borrows from both — bronze/silver/gold mirrors Inmon stages

Data Warehouse Technologies

Datawarehouse Type	Platforms
Appliance	    
Cloud Native	   



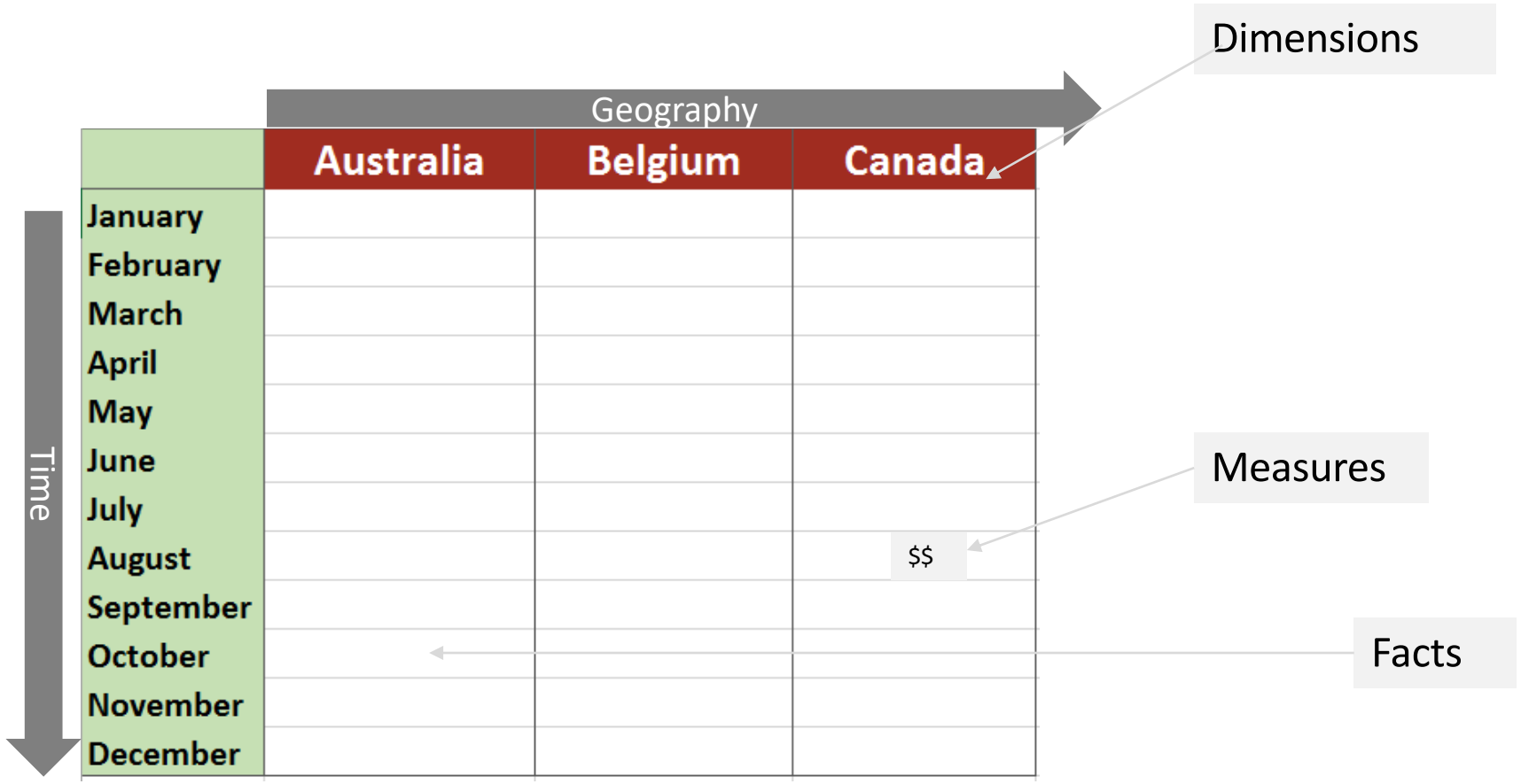
Dimensional Modeling

Module 3

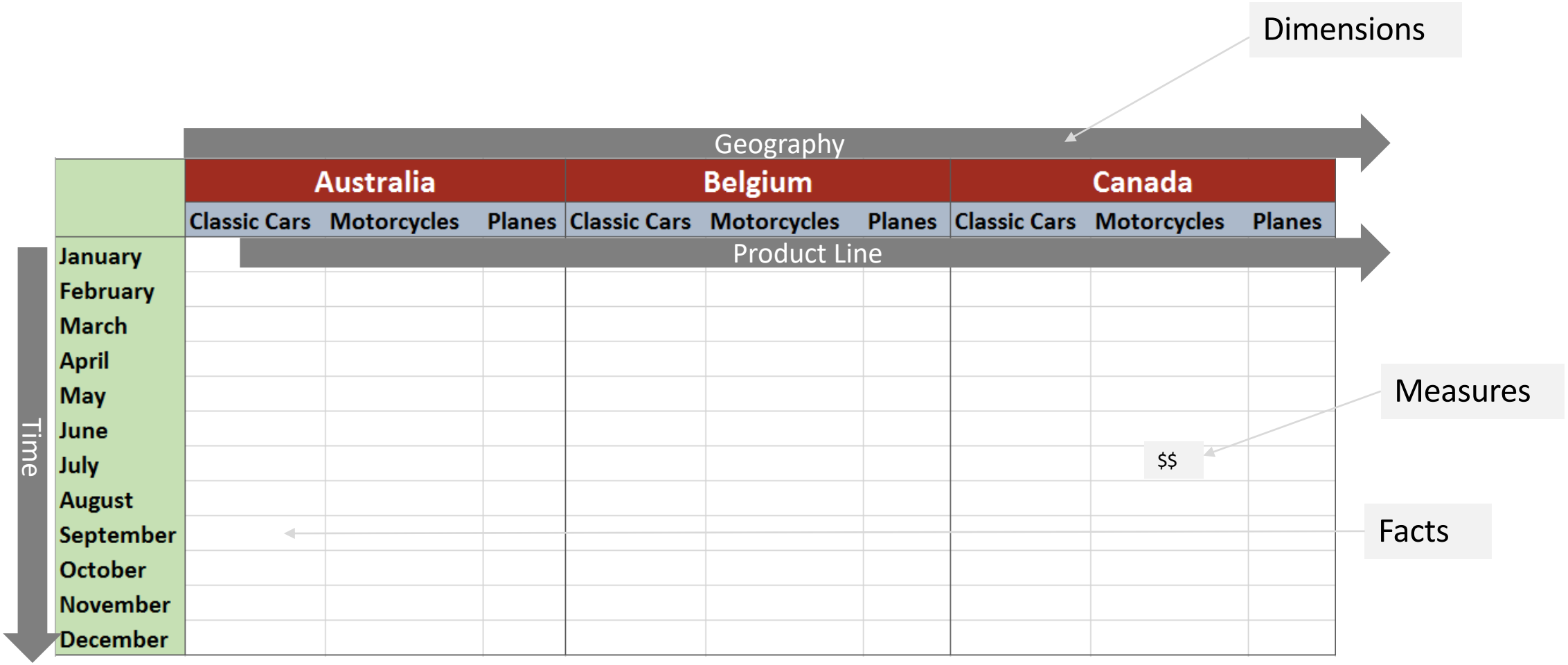
Dimensional Modeling Concepts

- Framework for organizing data for analytical queries; developed by Ralph Kimball
- Fact: a measurable business event (sale, click, payment) at a specific grain
- Dimension: descriptive context for a fact (who, what, when, where, why)
- Grain: the level of detail of each row in the fact table; declare it explicitly
- Dimensional models are denormalized intentionally — optimize for reads, not writes

Two-Dimensional Data

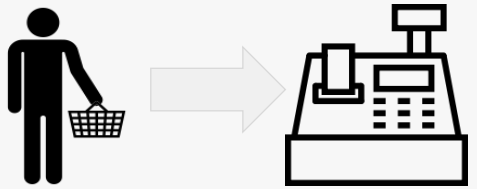


Three-Dimensional Data



Fact Tables

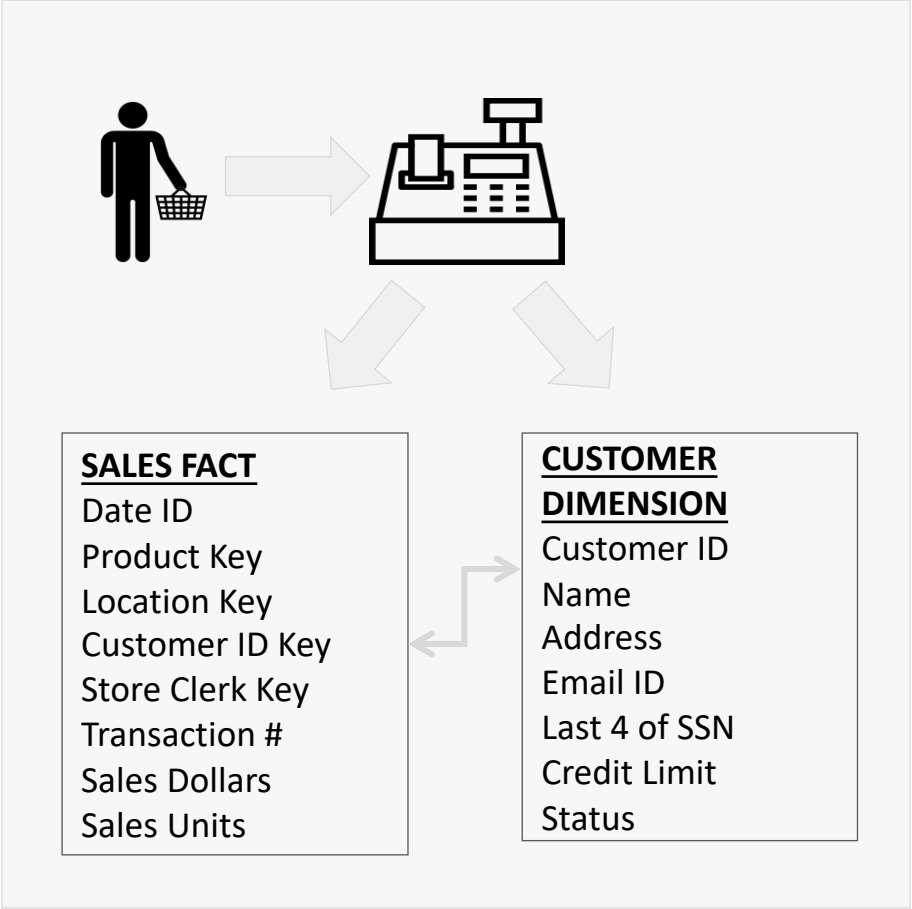
- Contains measures (numeric business metrics) and foreign keys to dimensions
- Additive facts: safe to sum across all dimensions (revenue, quantity)
- Semi-additive: sum across some dimensions but not all (account balance by date)
- Non-additive: never summed (ratios, percentages); store numerator + denominator
- Transactional, snapshot, and accumulating snapshot are common fact table types



SALES FACT
Date ID
Product Key
Location Key
Customer ID Key
Store Clerk Key
Transaction #
Sales Dollars
Sales Units

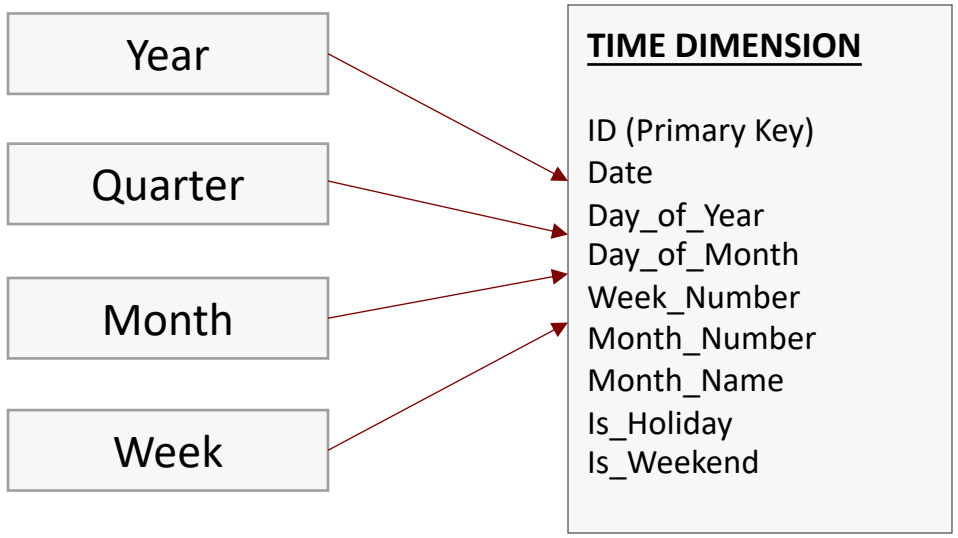
Dimension Tables

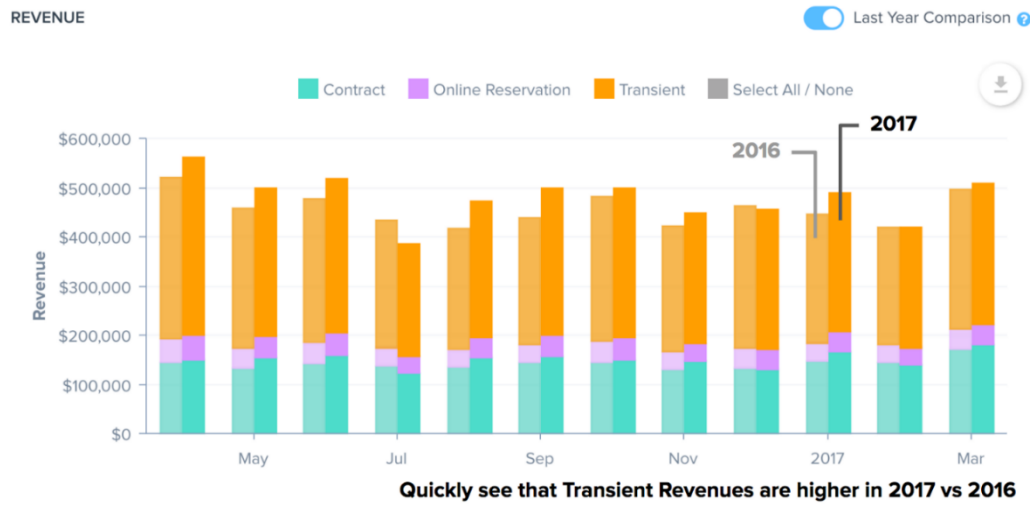
- Descriptive attributes that give context to fact table rows
- Date/Time dimension: pre-populated calendar table with fiscal periods, holidays
- Customer, Product, Store, Employee dimensions are common business examples
- Denormalize dimension hierarchies into one wide table (star) for query speed
- Junk dimension: low-cardinality flags and indicators grouped into one table



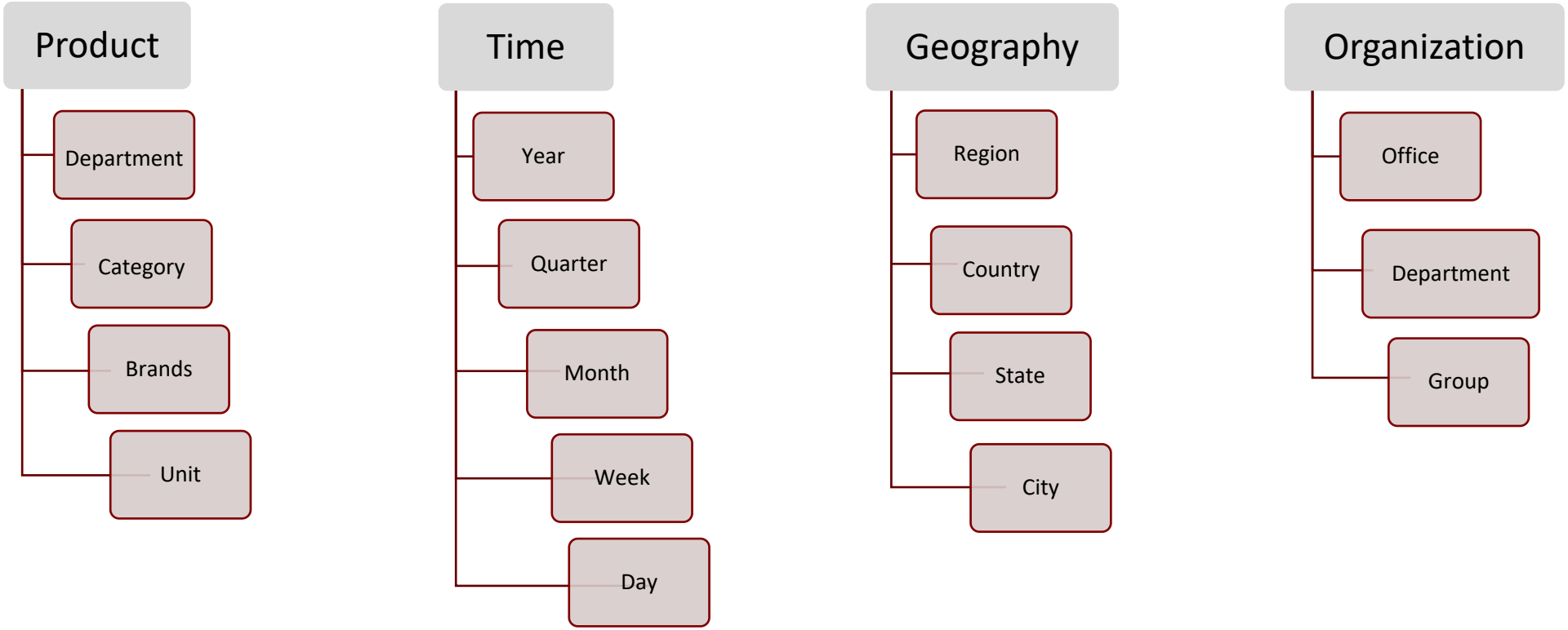
Date-Time Dimension

- Most business reporting leverages date/time analysis
  - Perform date time period comparisons
  - Track holidays and other significant dates
  - Useful in building time-based slicing and dicing - MTD, QTD, YTD

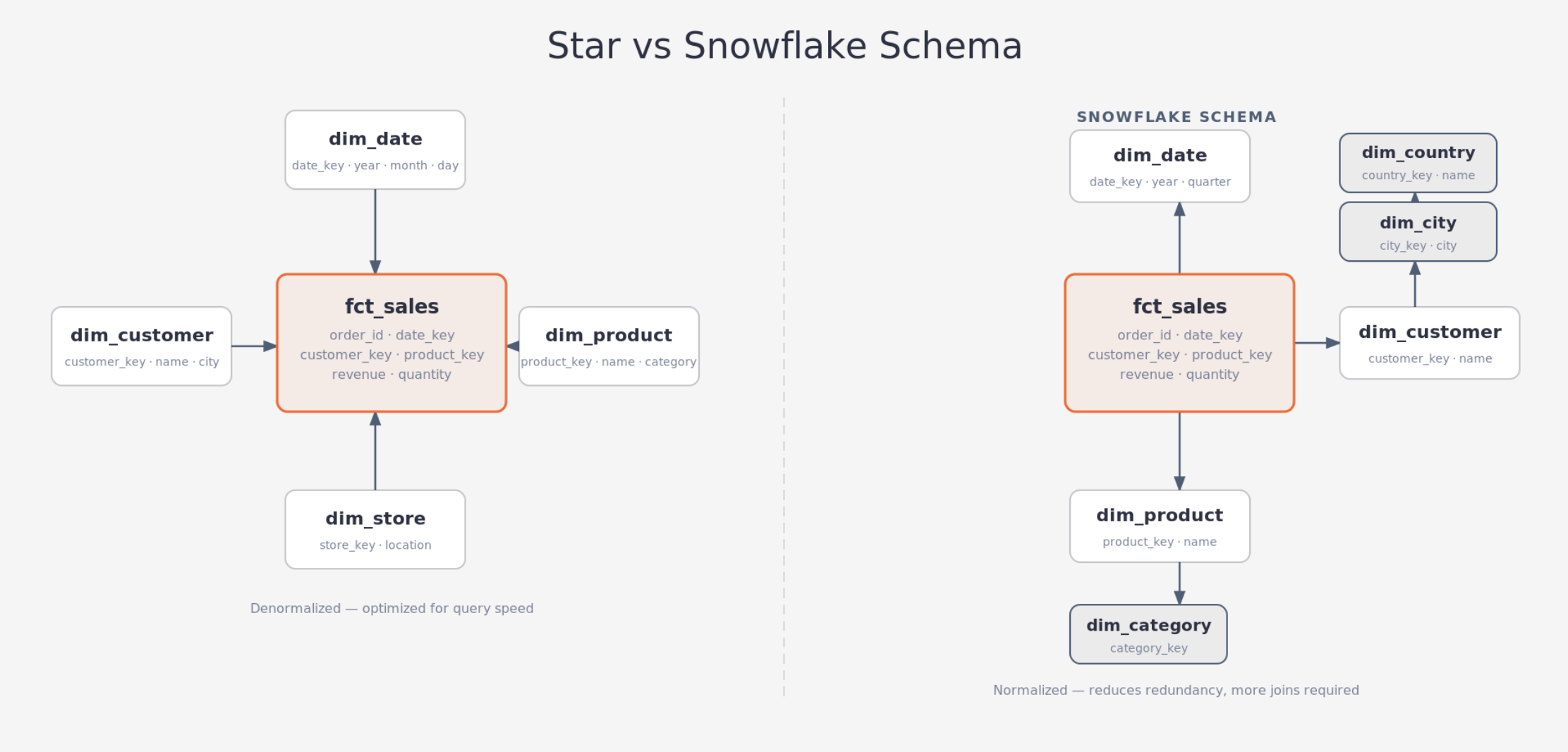




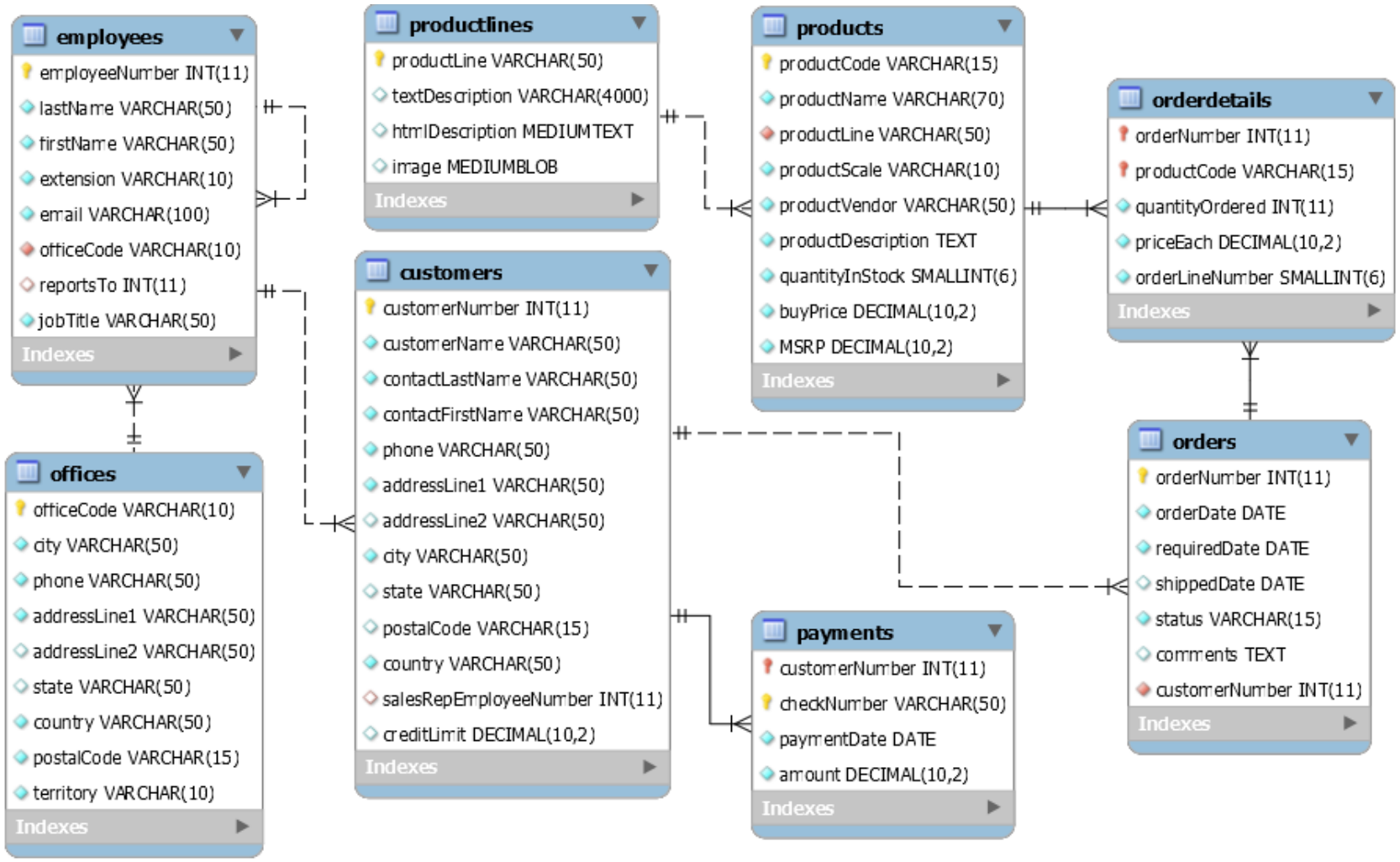
Dimension Hierarchies



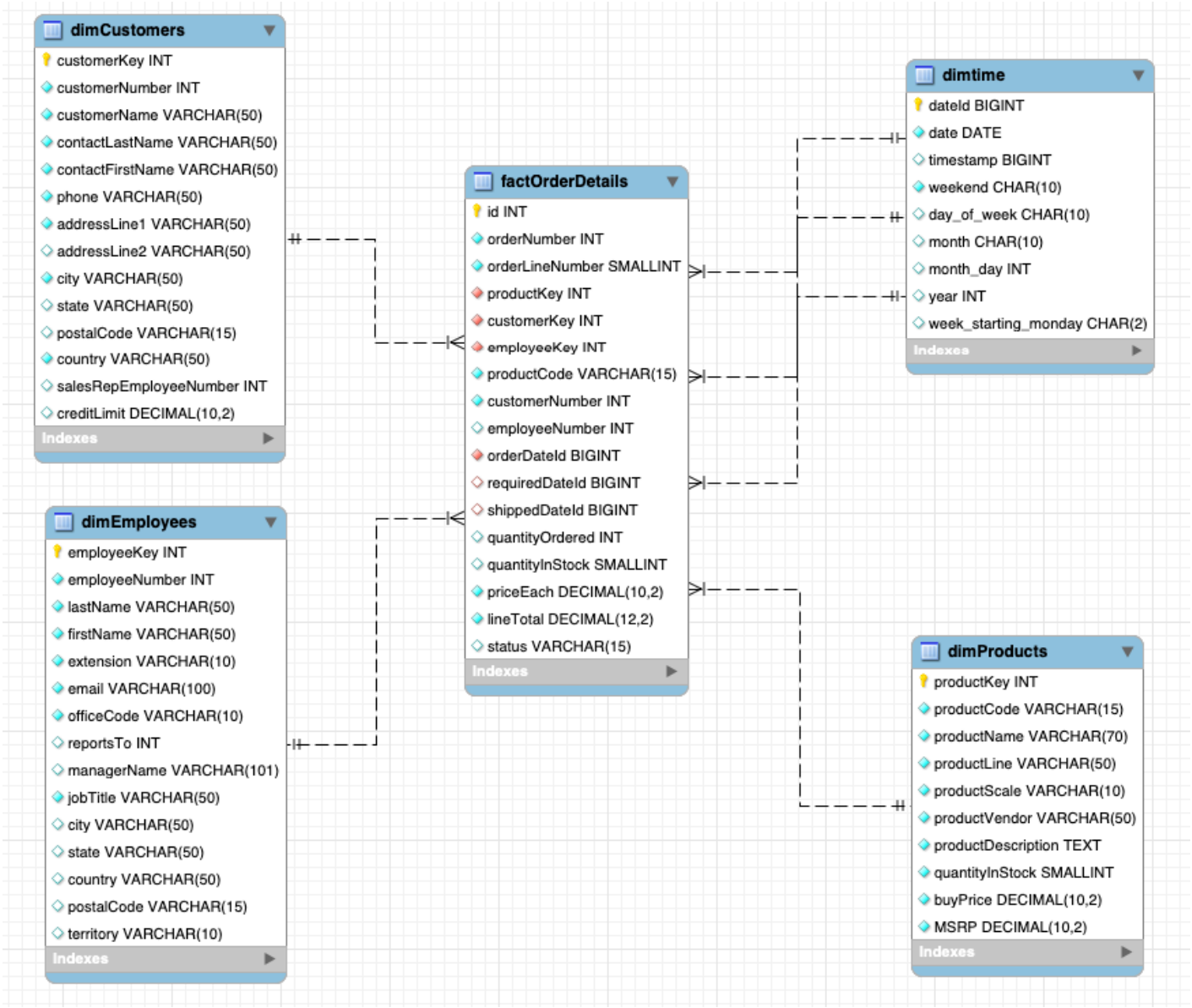
Star vs Snowflake Schema



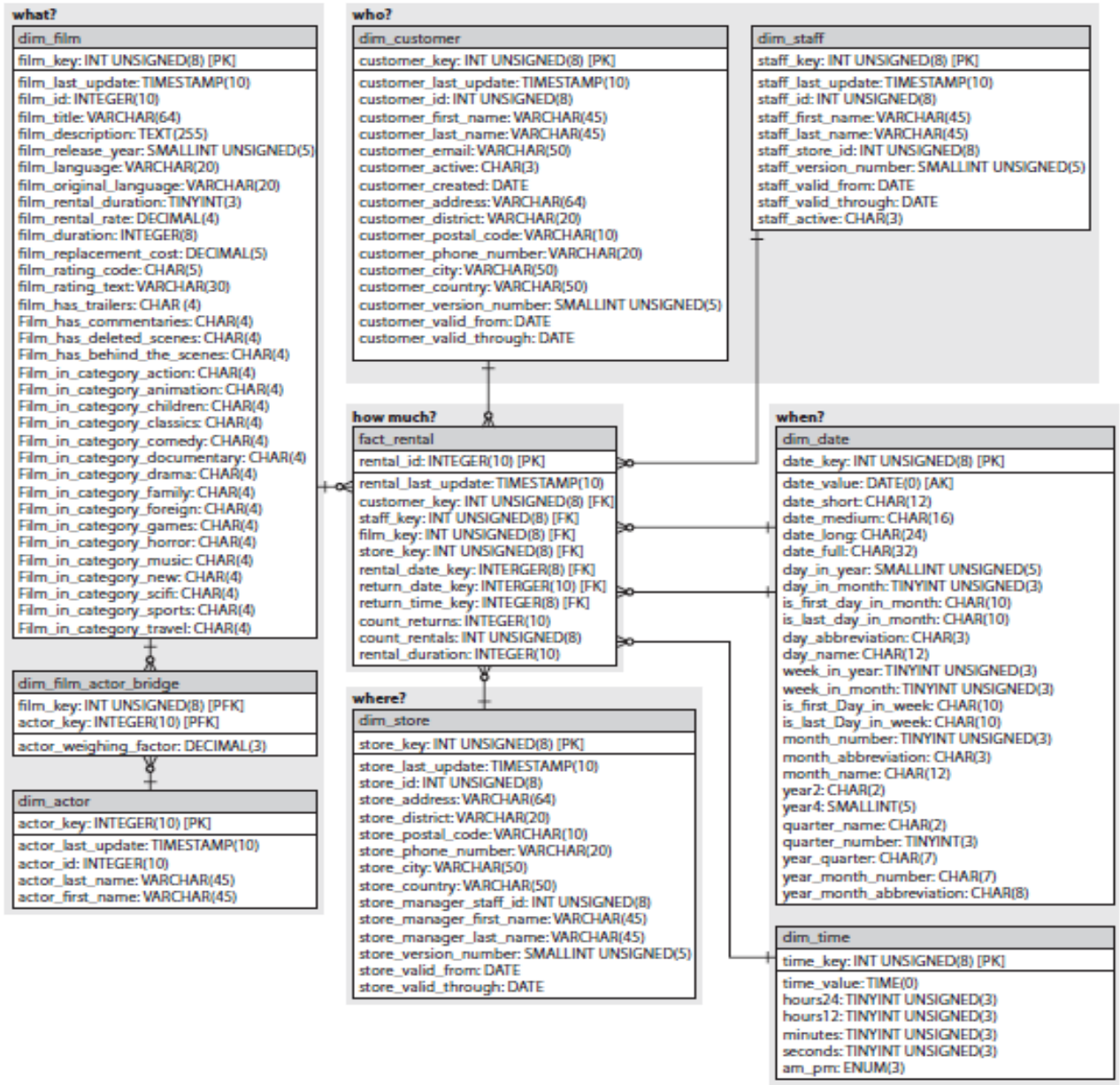
Classic Models ER Model



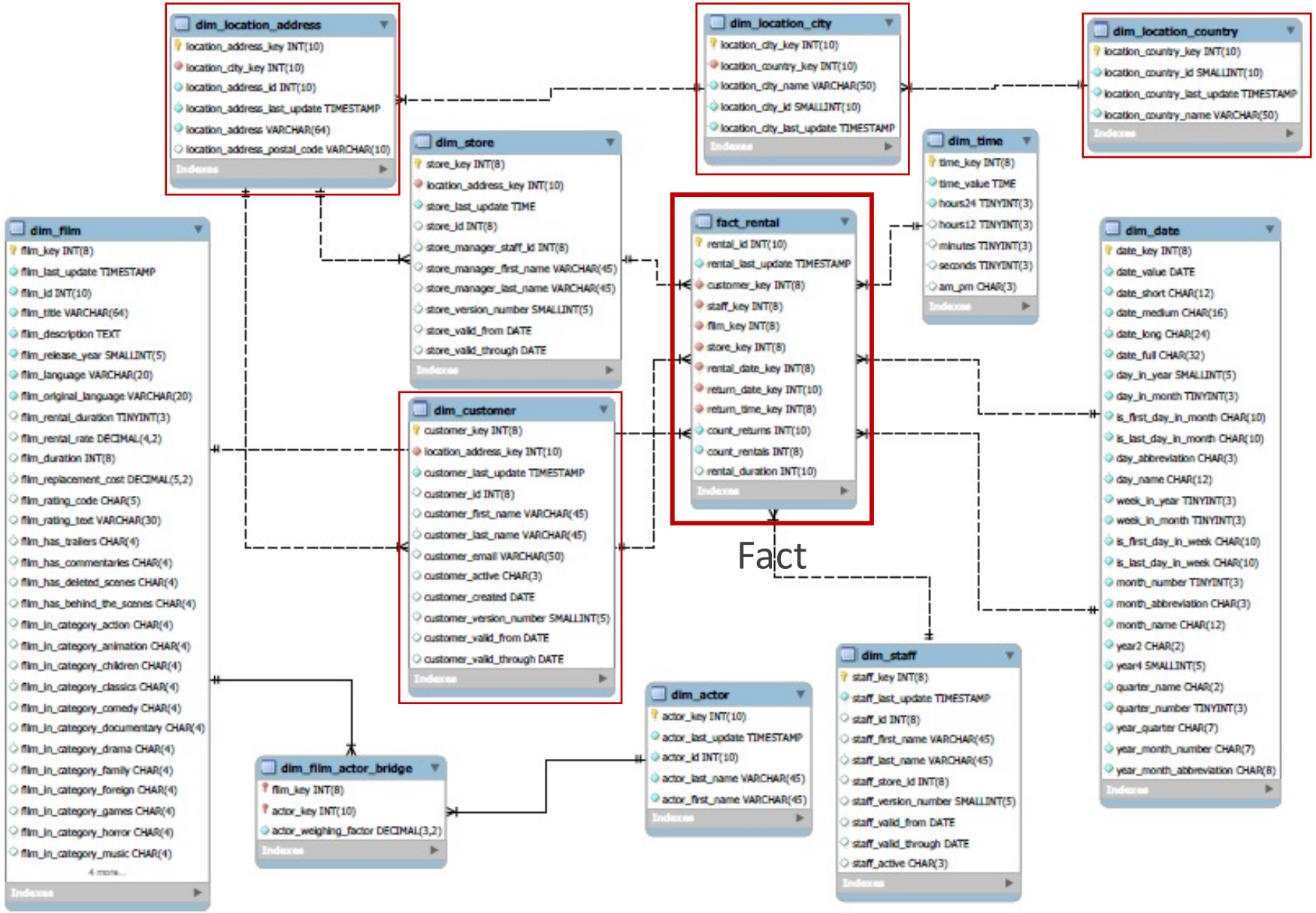
Classic Models Star Schema



Sakila
Star Schema

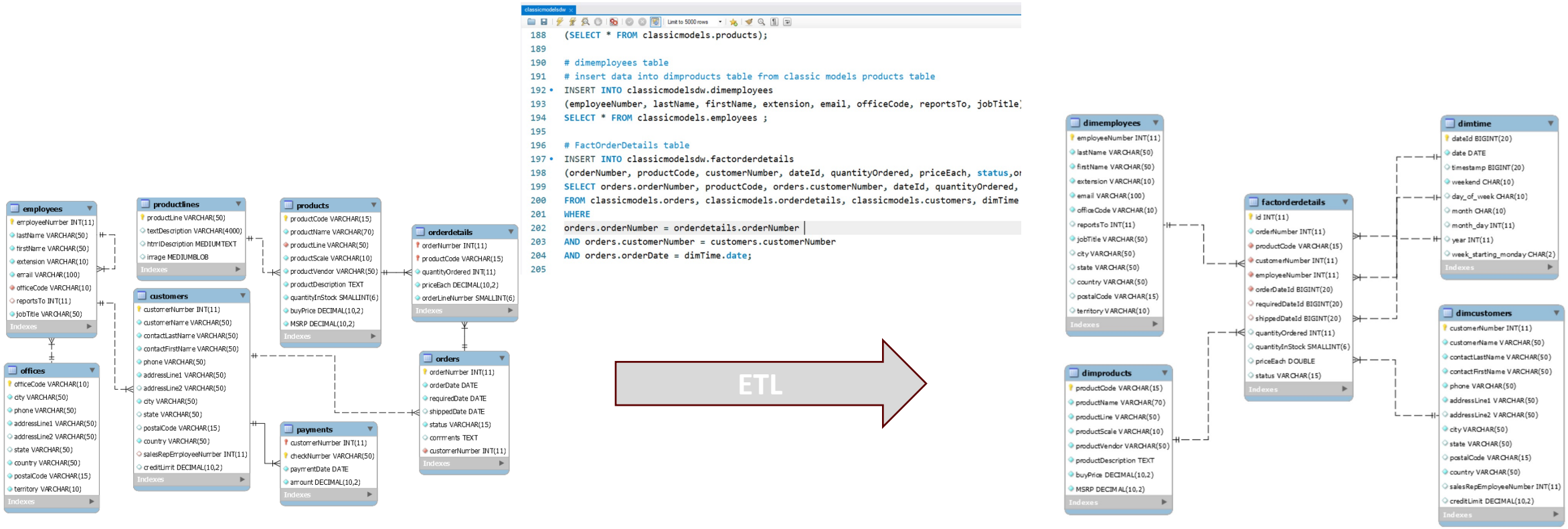


Sakila Snowflake

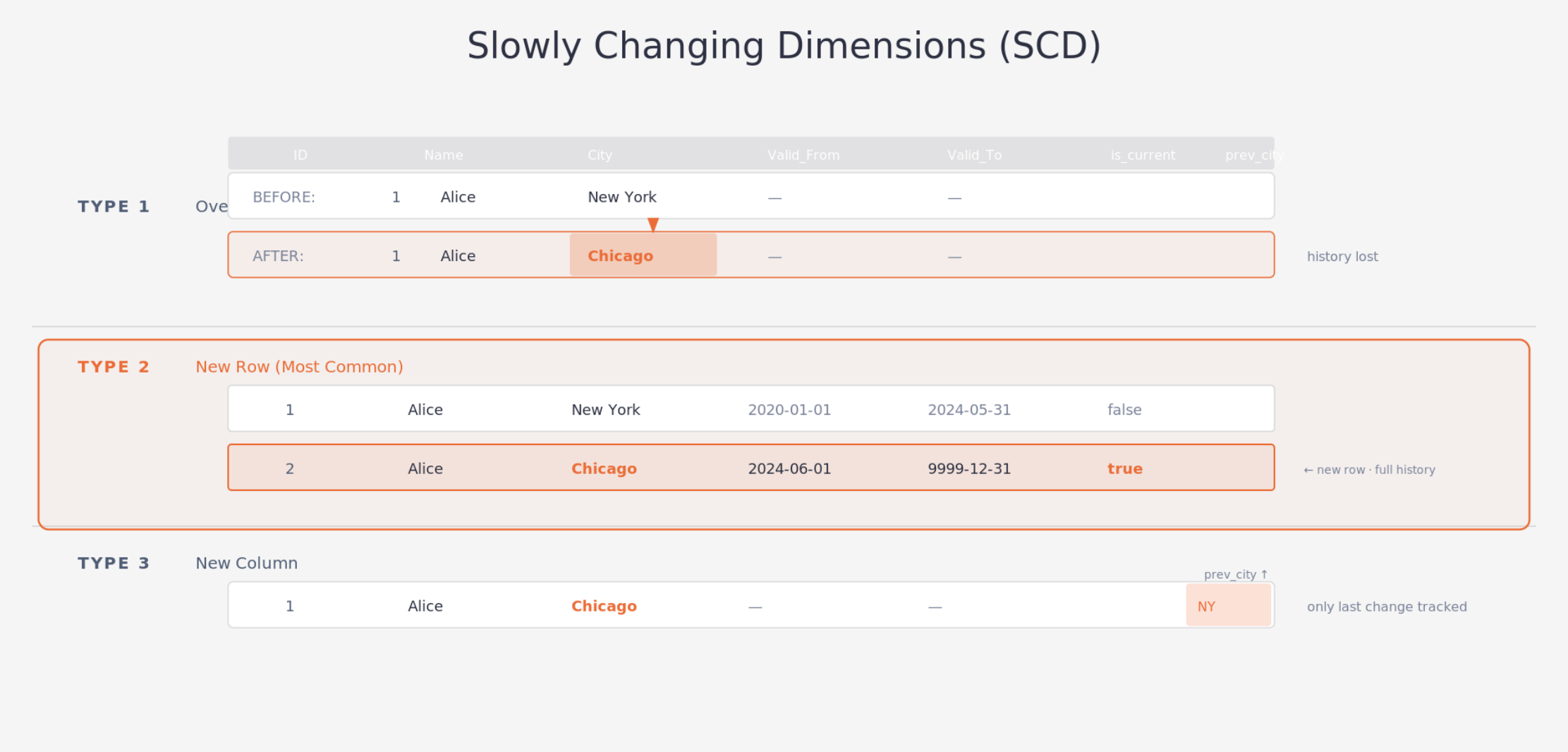


Exercise - Create Dimensional Model

- ETL from ClassicModels ER model to ClassicModels dimensional model.



Slowly Changing Dimensions



SCD : Type 1

- Changes the value in the dimension's record i.e. the new value replaces the old value.
- No history is preserved
- The simplest approach, used most often when a change in a dimension is the result of an error

CUSTOMER

CustomerKey	CustomerID	CustomerName	TaxBracket
1	111	Linda	Low
2	222	Susan	Medium
3	333	William	High

CUSTOMER

CustomerKey	CustomerID	CustomerName	TaxBracket
1	111	Linda	Low
2	222	Susan	High
3	333	William	High

SCD : Type 2

- Creates a new additional dimension record using a new value for the surrogate key every time a value in a dimension record changes and used in cases where history should be preserved
- Can be combined with the use of timestamps and row indicators
  - Timestamps - columns that indicates the time interval for which the values in the records are applicable
  - Row indicator - column that provides a quick indicator of whether the record is currently valid

CUSTOMER

CustomerKey	CustomerID	CustomerName	TaxBracket
1	111	Linda	Low
2	222	Susan	Medium
3	333	William	High

CUSTOMER

CustomerKey	CustomerID	CustomerName	TaxBracket
1	111	Linda	Low
2	222	Susan	Medium
3	333	William	High
4	222	Susan	High

SCD : Type 2

CUSTOMER

CustomerKey	CustomerID	CustomerName	TaxBracket
1	111	Linda	Low
2	222	Susan	Medium
3	333	William	High

CUSTOMER

CustomerKey	CustomerID	CustomerName	TaxBracket	Effective StartDate	Effective EndDate	Row Indicator
1	111	Linda	Low	1.1.2000	n/a	Current
2	222	Susan	Medium	1.1.2000	12.31.2007	Not Current
3	333	William	High	1.1.2000	n/a	Current
4	222	Susan	High	1.1.2008	n/a	Current

SCD : Type 3

- Involves creating a “previous” and “current” column in the dimension table for each column where changes are anticipated
- Applicable in cases in which there is a fixed number of changes possible per column of a dimension, or in cases when only a limited history is recorded.
- Use of timestamps

CUSTOMER

CustomerKey	CustomerID	CustomerName	TaxBracket
1	111	Linda	Low
2	222	Susan	Medium
3	333	William	High

CUSTOMER

CustomerKey	CustomerID	CustomerName	Previous TaxBracket	Current TaxBracket
1	111	Linda	n/a	Low
2	222	Susan	Medium	High
3	333	William	n/a	High

CUSTOMER

CustomerKey	CustomerID	CustomerName	Previous TaxBracket	Previous TaxBracket EffectiveDate	Current TaxBracket	Current TaxBracket EffectiveDate
1	111	Linda	n/a	n/a	Low	1.1.2000
2	222	Susan	Medium	1.1.2000	High	1.1.2008
3	333	William	n/a	n/a	High	1.1.2000

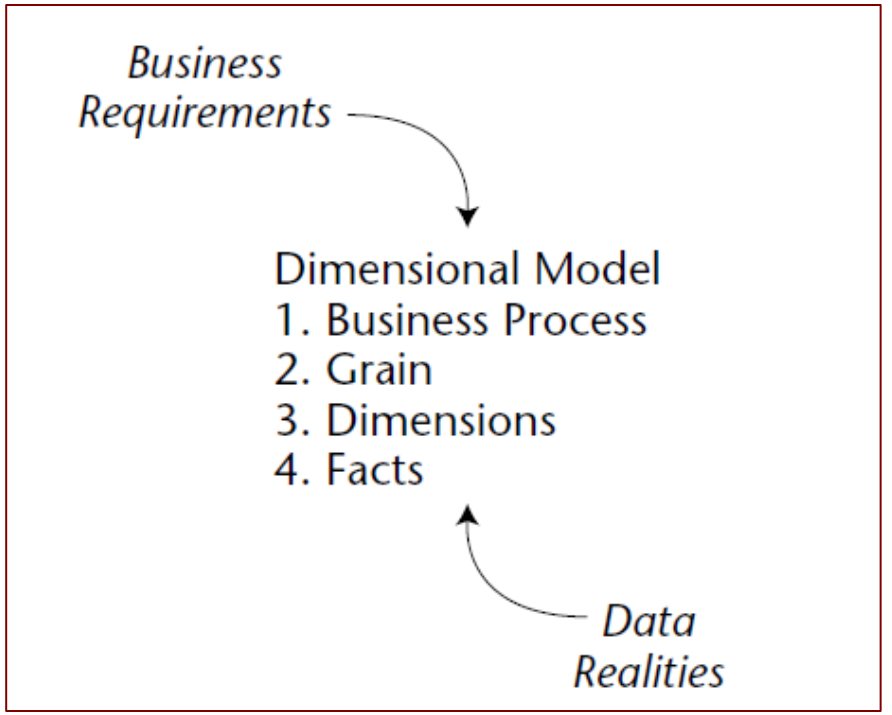
Dimensional Design Process

- Step 1: Select the business process to model (e.g., online sales)
- Step 2: Declare the grain (e.g., one row per order line item)
- Step 3: Identify dimensions that apply at that grain
- Step 4: Identify numeric facts that belong at that grain
- Validate grain: every fact must make sense at the declared level of detail

Dimensional Design Process

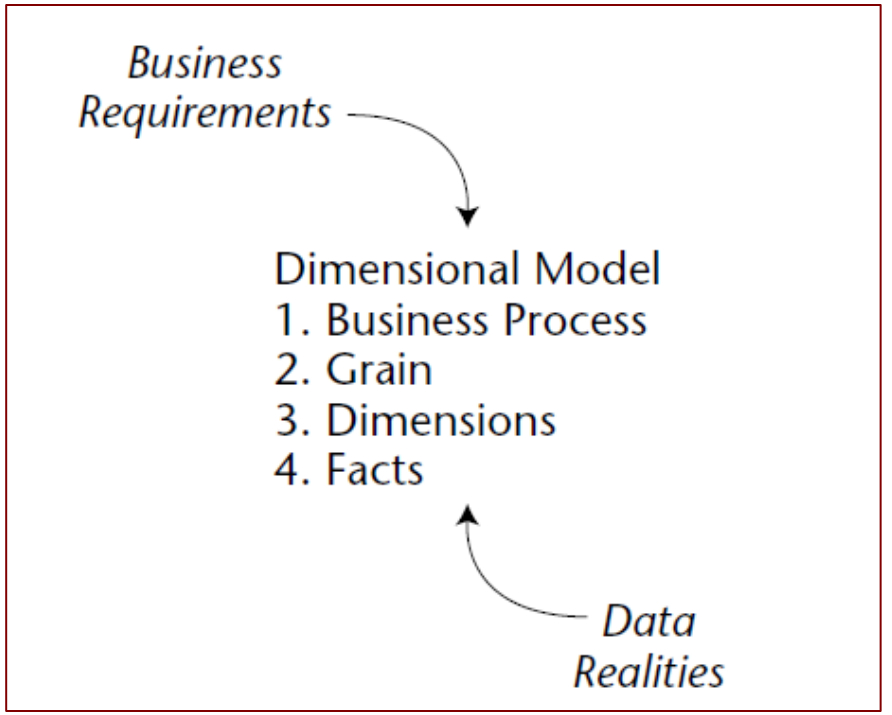
1. Select the business process to model
   - Business process is a set of related activities (customers, profit, sales, organizations, and products)
   - Rather than business department or function (E.g., purchasing, ordering, shipping, invoicing, inventorying)

2. Declare the grain of the business process
   - Specifies individual fact table row
   - E.g., individual line item on sales ticket, daily snapshot of the inventory levels for a product



Dimensional Design Process

3. Choose the dimensions that apply for each fact table row.
   - How does the business describe the data that results from the business process? E.g., date, product, store, customer, transaction type
4. Identify the numeric (measured) facts that will populate each fact table row.
   - What are we measuring? Typical facts are numeric additive figures E.g., quantity ordered, dollar cost amount
   - Consider both user requirements as well as realities of the source data



Advanced Dimensional Patterns

Pattern	Purpose	Example	Watch out
Bridge table	Many-to-many fact to dimension	Sale with several promotions	Complicates queries; use sparingly
Role-playing dimension	One dimension, several roles	DIM_DATE as order, ship, delivery date	Expose each role as an aliased view
Factless fact table	Records events or coverage	Student attendance	Count rows; no measures
Conformed dimension	Shared across facts and marts	DIM_CUSTOMER in sales and support	One owner; enables drill-across
Aggregate table	Pre-computed coarser rollup	Monthly sales by region	Keep in sync with atomic fact

Case Study 1 : Retail

- Design a dimensional model for a global retailer where each store has the following:
  - Departments : grocery, frozen foods, dairy, meat, produce, bakery, floral, health/beauty aids, etc.
  - > 60,000 products (SKUs = stock keeping units) on shelves
  - > 55,000 SKUs with UPCs
  - > 5,000 SKUs without UPCs but with assigned SKU numbers
- Data Collection:
  - From cash registers into a point-of-sale (POS) system
  - At back door where vendors make deliveries



Management Concerns

- Maximizing profit
- Logistics of ordering, stocking, and selling products
- Product pricing
- Use of promotions to increase sales
  - Temporary Price Reductions (TPR)
  - Advertisements (newspapers, magazines…etc.)
  - Grocery store displays
  - Coupons

Step 1 : Business Process

- Decide what business process to model, by combining an understanding of the business requirements with an understanding of data realities.
- The first dimensional model built should be the one
  - with the most impact,
  - that answers the most pressing business questions,
  - is readily accessible for data extraction

POS Retail Sales

Insight : What products are selling in which stores on what days and under what promotional conditions?

Step 2 : Granularity

- What level of data detail should be made available in the dimensional model?
- Choose the most atomic information captured by the business process.
  - Most detailed, cannot be subdivided
  - Facilitates ad hoc, unexpected usage and ability to drill down to details

Sales

individual line item on a POS transaction

Step 3 – Dimensions

- A careful grain statement determines the primary dimensions.
- Add additional dimensions based on new business requirements.
- If an additional desired dimension violates the grain by causing additional fact rows to be generated, then the grain statement must be revised to accommodate this dimension.

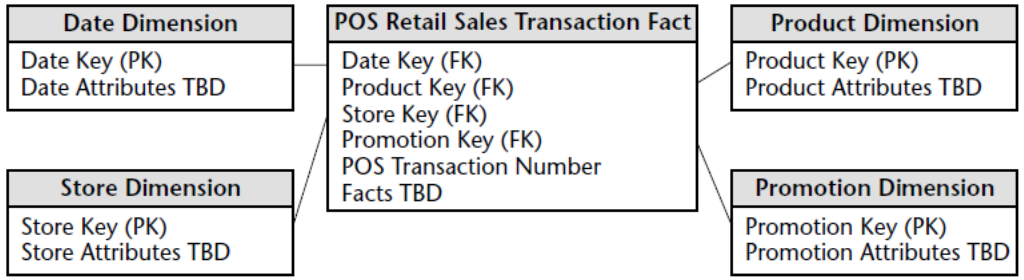
POS Retail Sales

Date, Product, Store, Promotion

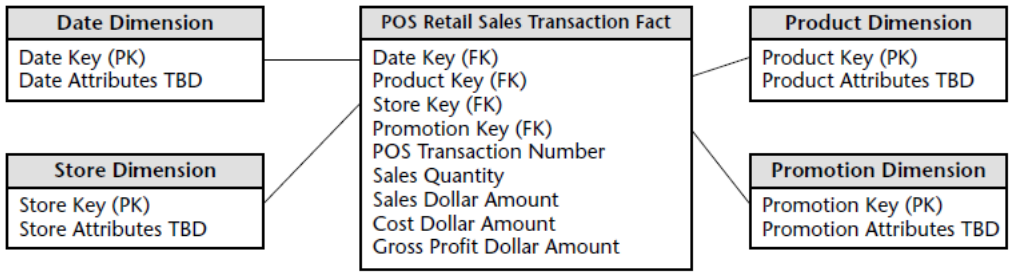
Step 4 : Facts

- Picking the business measurements (grain) for the fact table
- Facts collected by POS system:
  - Sales quantity, sales price/unit, sales $ amount, standard cost $ amount
  - Gross Profit = cost – sales
    - Recommendation: Include in fact table even though it can be calculated. Eliminates the possibility of user error.
  - For non-additive measurements such as percentages and ratios (e.g., gross margin) store the numerator (gross profit) and denominator ($ revenue) in the fact table. The ratio can be calculated in a data access tool for any slice of the fact table.
  - Caution: Calculate the ratio of the sums, not the sum of the ratios

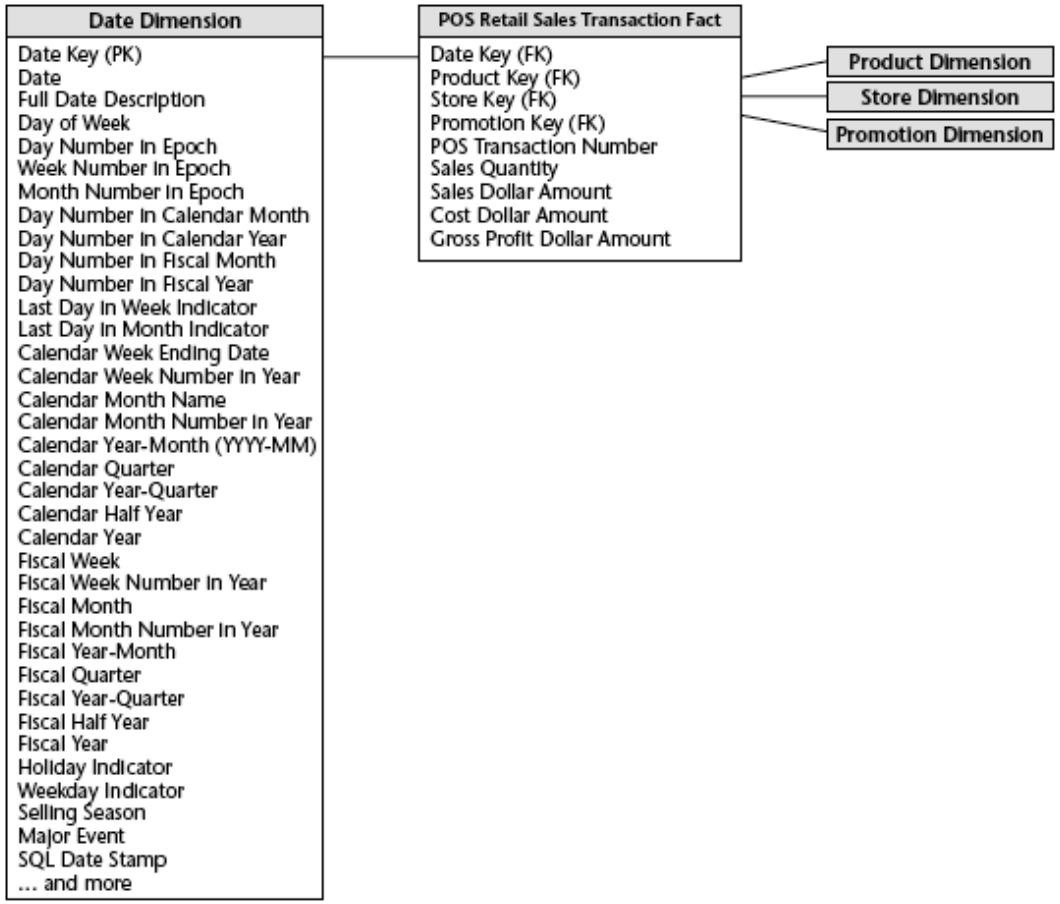
Preliminary Retail Sales Schema



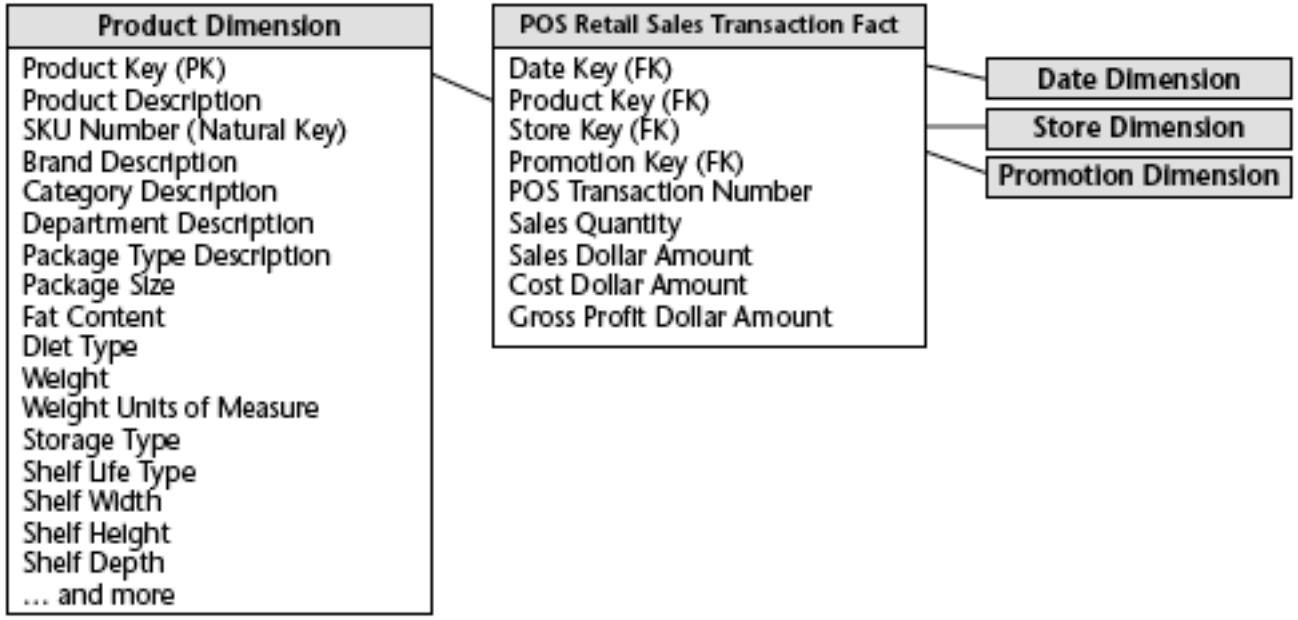
Measured Facts in Retail Sales Schema



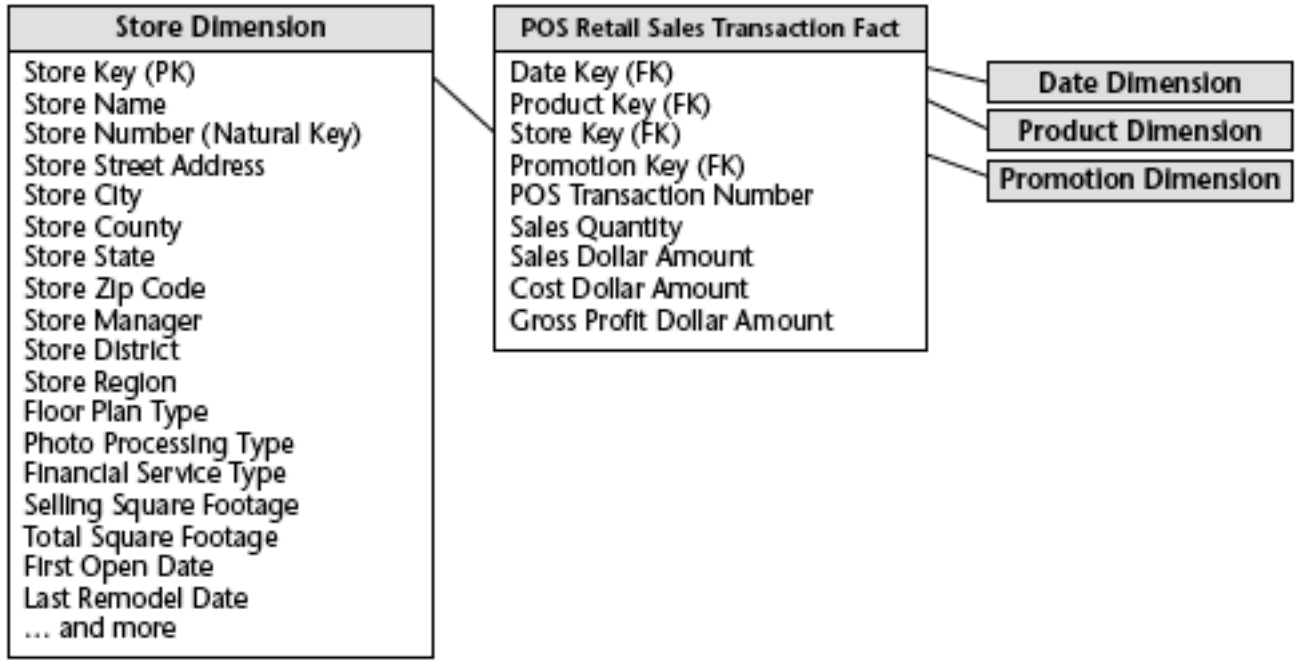
Date Dimension



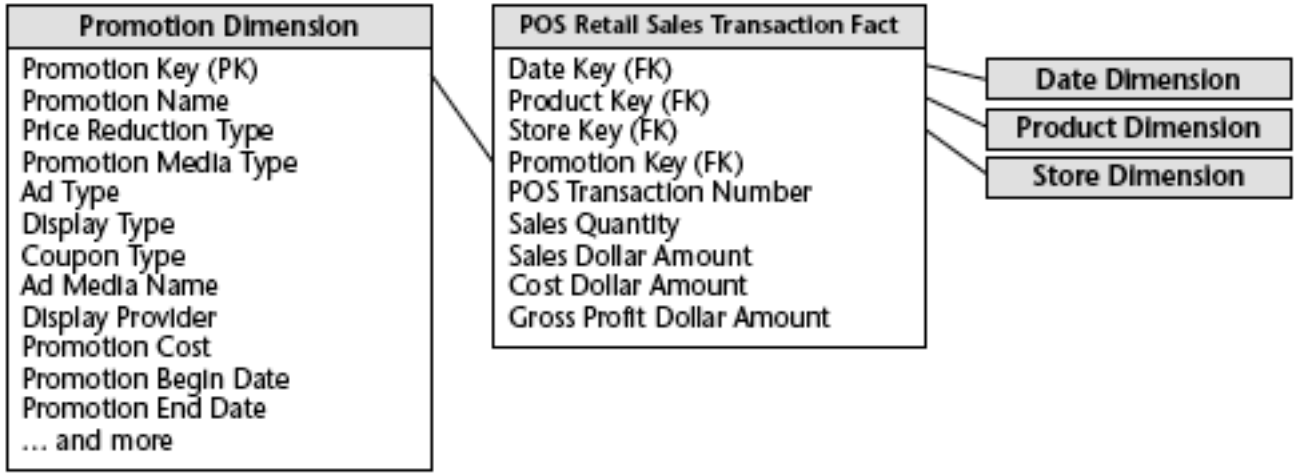
Product Dimension



Store Dimension



Promotion Dimension

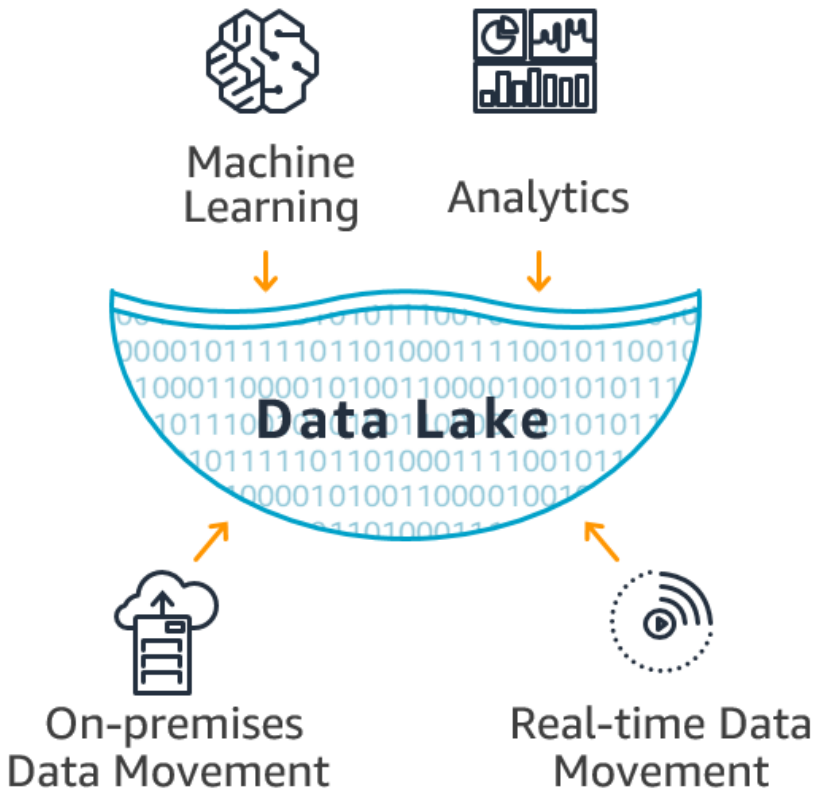


Lakehouse Architecture and Governance

Module 4

Data Lake

- Storage repository that holds a vast amount of raw data in its native format
- Data Lakes retain all data
- Data Lakes support all data types
- “Schema on Read” – schema requirements are not defined until the data is needed.
- Data swamp risk without governance



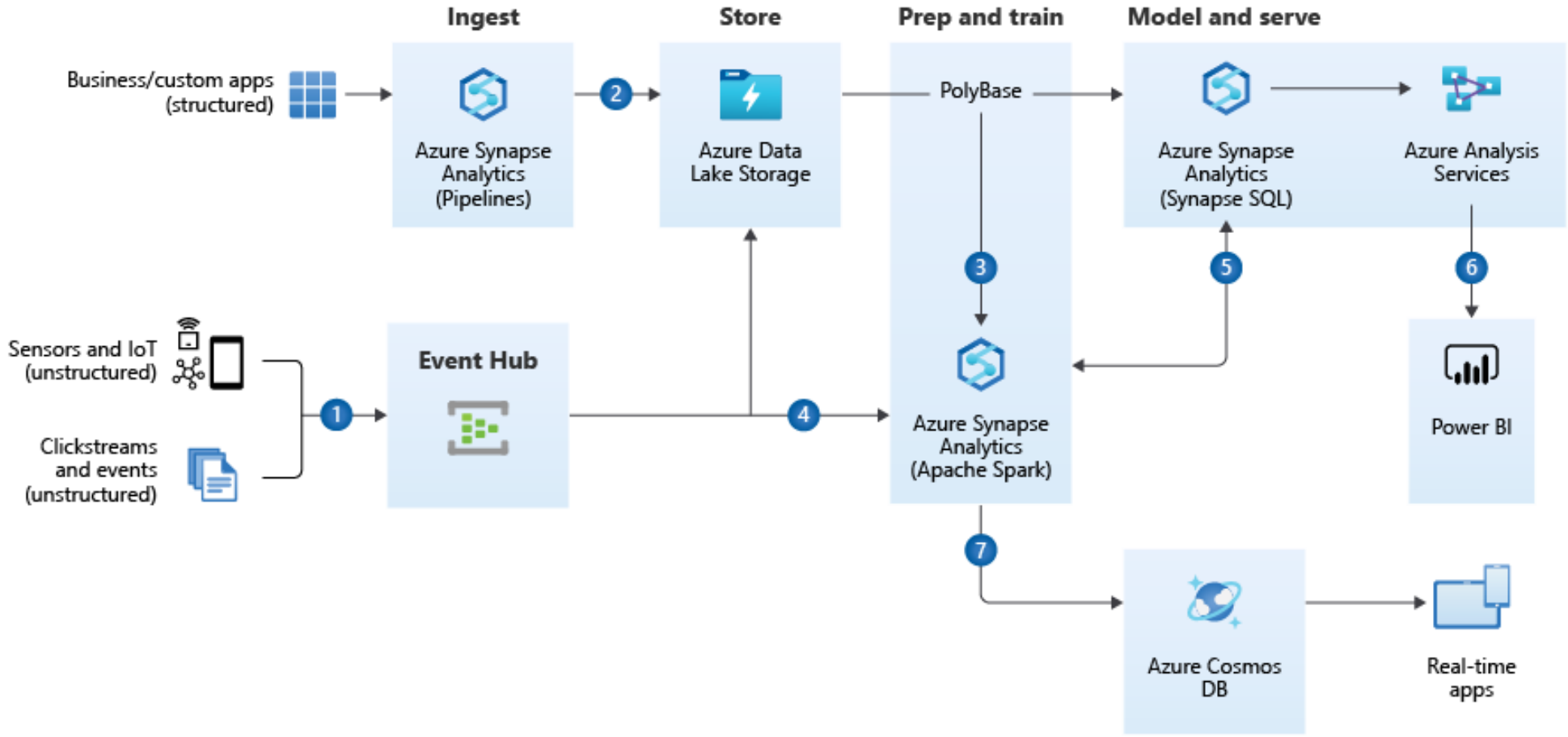
Cloud Data Lake Platforms







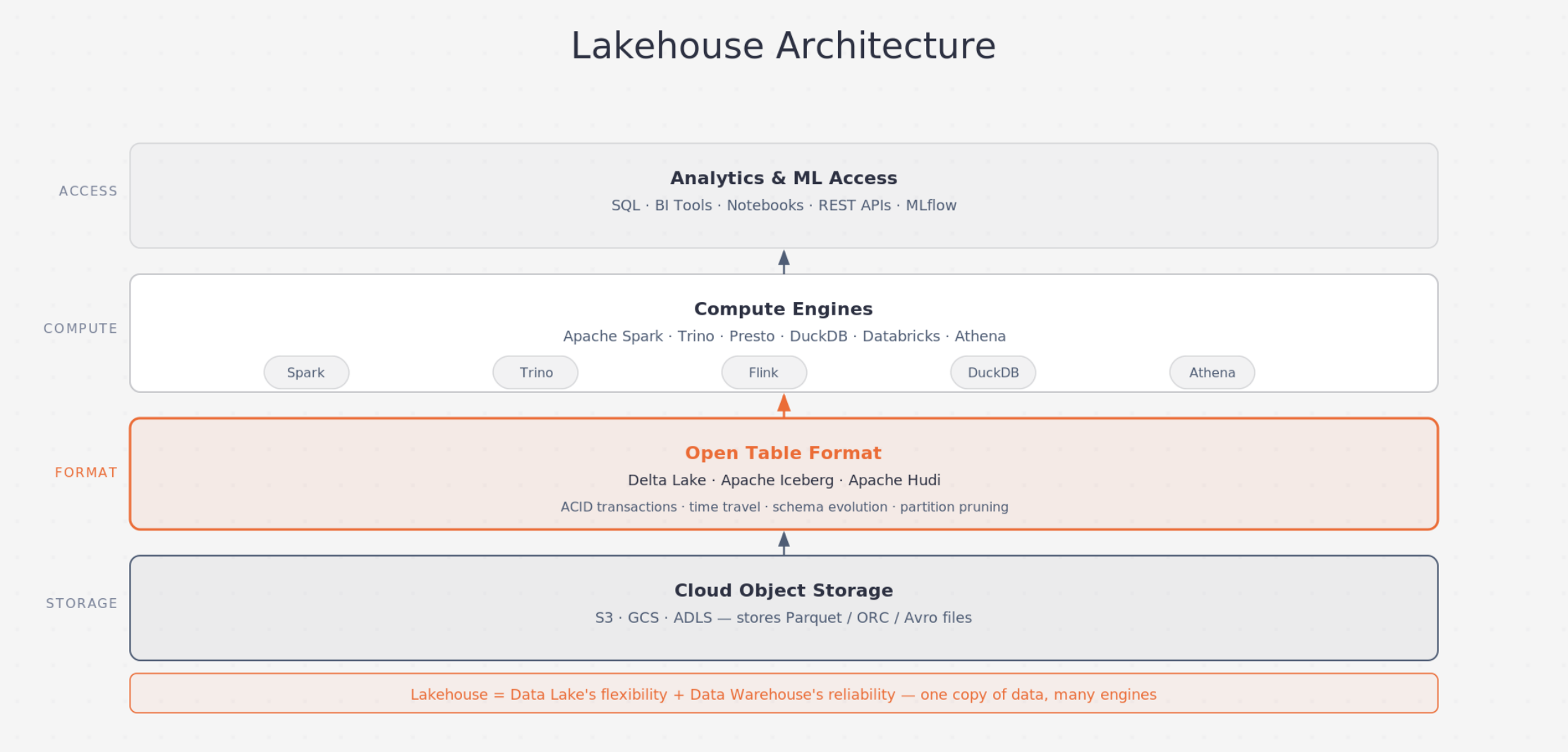
Azure Data Lake Pipeline



Data Warehouse vs Data Lake

Attribute	Data Warehouse	Data Lake
Key Benefit	Provides a single enterprise-wide view of data from multiple sources	Allows usage of raw structured and unstructured data from a centralized low-cost store
Scale	Scales to large volumes at moderate cost - limited number of server nodes (Terabytes)	Scales to huge volumes at low cost - tens of thousands of storage and compute nodes (Petabytes)
Access methods	Standardized SQL and BI tools	SQL-like systems, big data, other programs
Workloads	Analytics- BI, Reporting	Batch and Stream processing, Machine Learning/AI, Data storage, backup and archiving
Users	Reporting, Business Intelligence users	Analytics, Data Scientists, Data Engineers
New content	Time consuming to design data model	Fast ingestion of new data/content
Schema	Schema on write (predefined schemas)	Schema on read (no predefined schemas)
Cost/efficiency	Efficiently uses CPU/IO. Storage can be expensive due to custom hardware and data replication	Efficiently uses storage capabilities at very low cost. Processing not inherent to the data lake

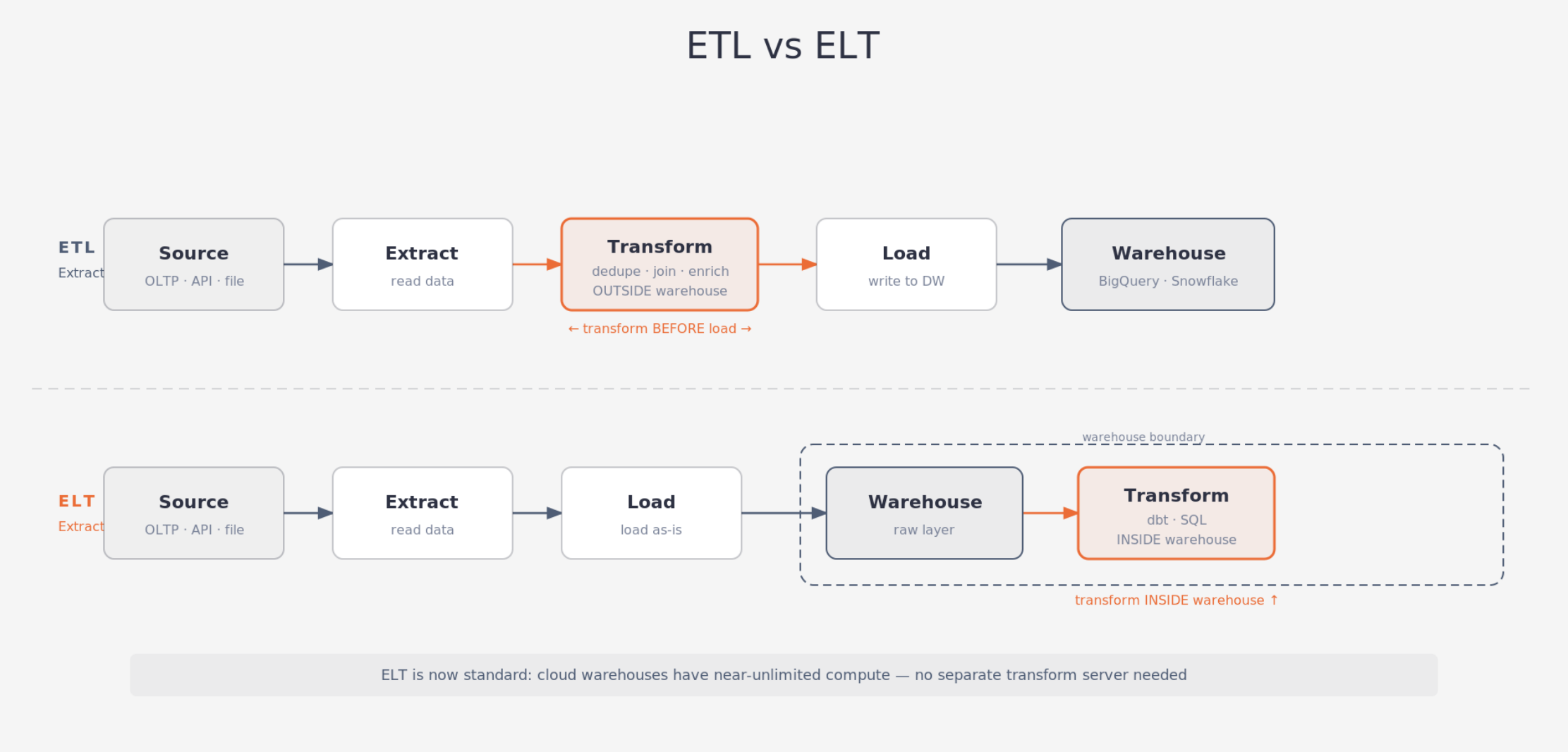
Lakehouse Architecture



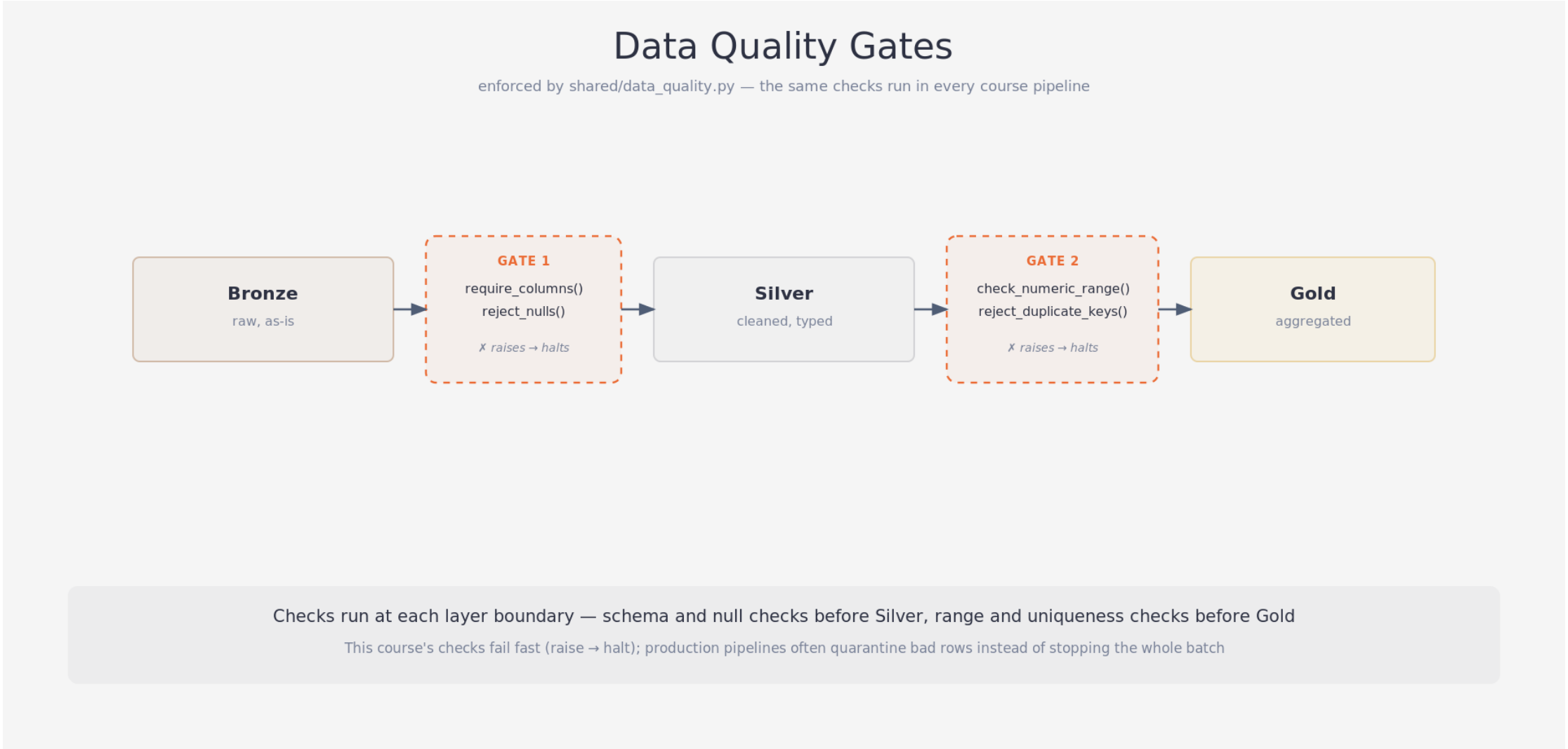
Bronze / Silver / Gold Layers



ETL vs ELT


ETL vs ELT
ETL
Extract
Source
OLTP · API · file
Extract
read data
Transform
dedupe · join · enrich
OUTSIDE warehouse
Load
write to DW
Warehouse
BigQuery · Snowflake
← transform BEFORE load →
ELT
Extract
Source
OLTP · API · file
Extract
read data
Load
load as-is
warehouse boundary
Warehouse
raw layer
Transform
dbt · SQL
INSIDE warehouse
transform INSIDE warehouse ↑
ELT is now standard: cloud warehouses have near-unlimited compute — no separate transform server needed

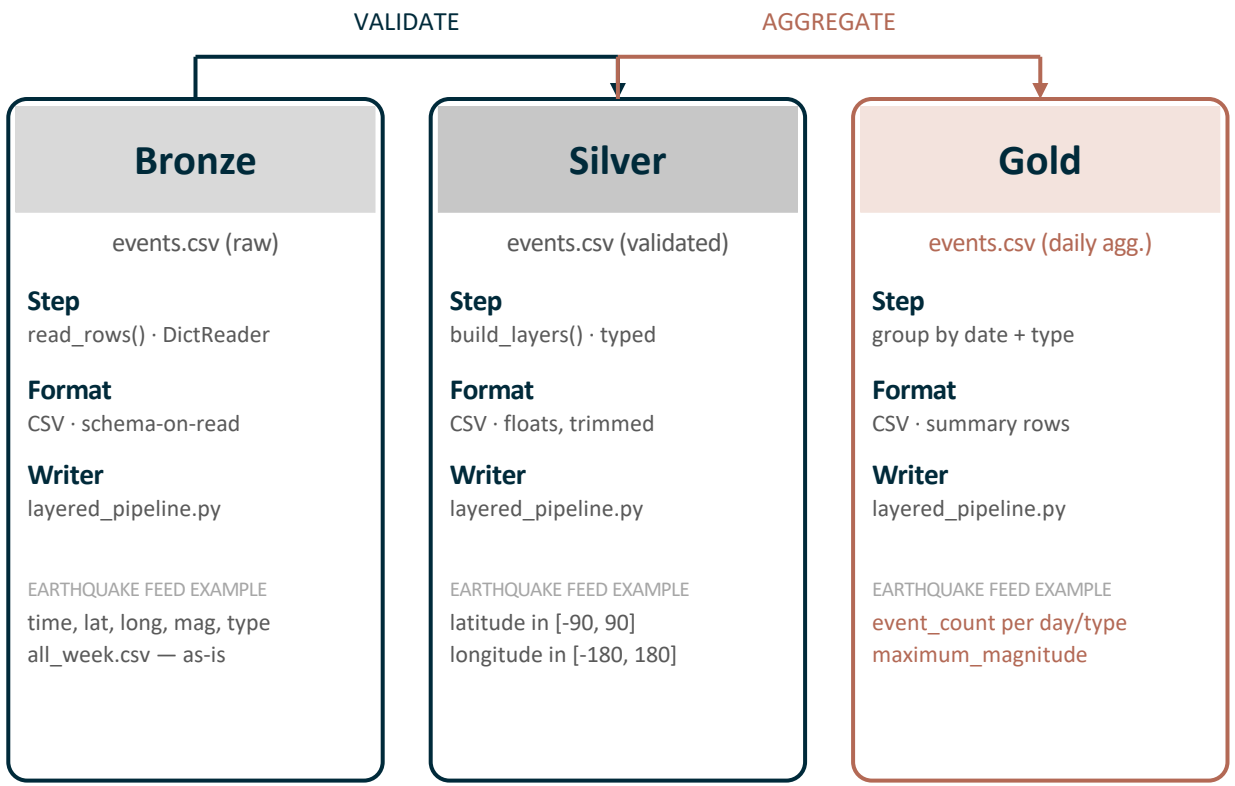
Data Quality Gates and Testing



Columnar Formats and Loading

- Parquet: open columnar format; excellent compression; predicate pushdown
- ORC: columnar; optimized for Hive; strong compression ratios
- Delta Lake adds ACID, DML, and versioning on top of Parquet files
- Loading strategies: COPY INTO, BigQuery Load Job, Storage Write API
- Compaction: merge many small files into fewer large files for query performance

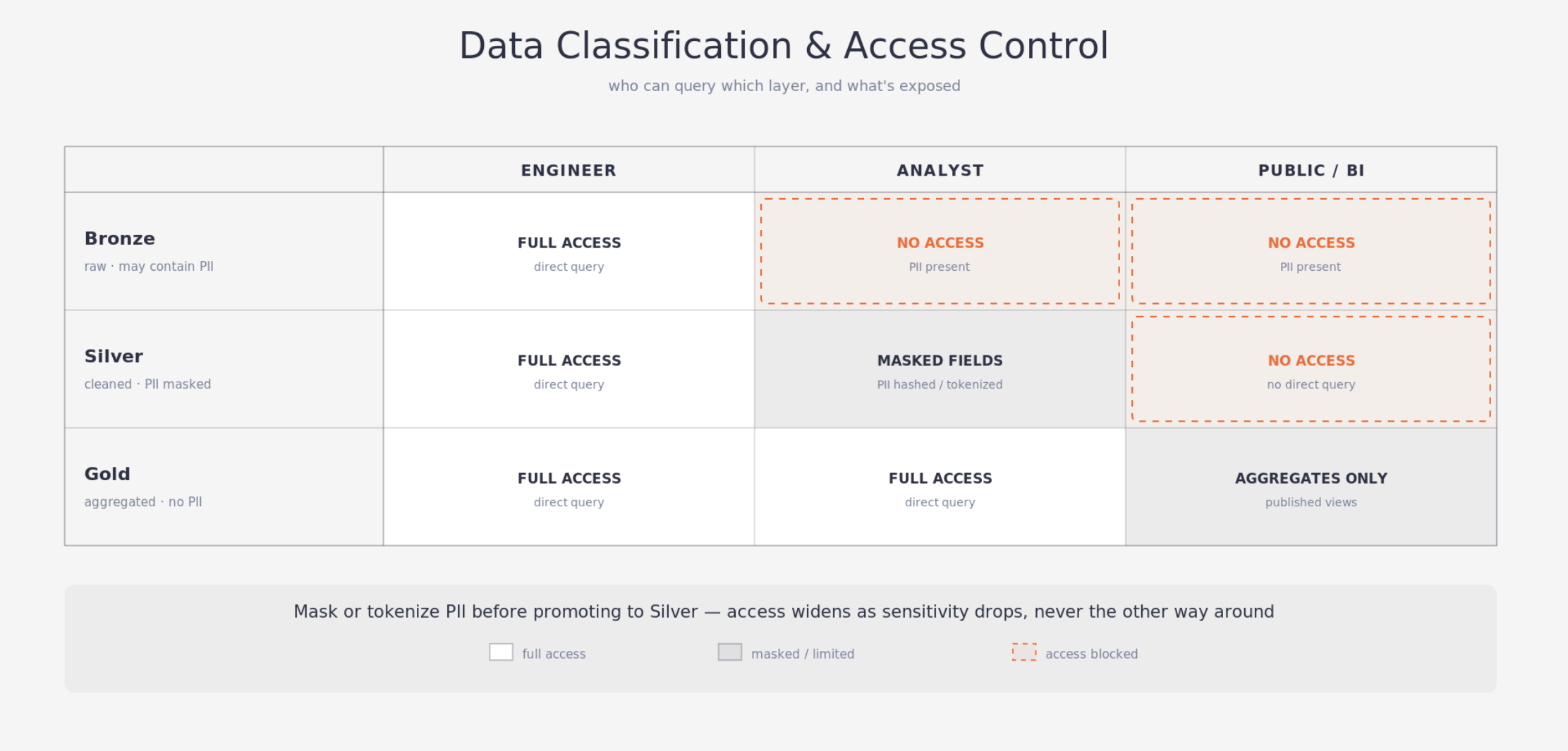
Layered Pipeline: Earthquake Data Walkthrough



Data Catalog and Lineage

- Data catalog: searchable inventory of datasets, schemas, owners, and descriptions
- Unity Catalog (Databricks), AWS Glue Catalog, GCP Dataplex, Collibra
- Column-level security: mask or redact sensitive fields based on user role
- Lineage: track column-level data flow from source to Gold layer and dashboards
- Audit logging: record every query, who ran it, when, and what data was accessed

Data Classification and Access Control



Open Table Formats

Format	Design	Strength	Typical use
Apache Iceberg	Metadata and manifests over Parquet	Hidden partitioning; schema evolution	Spark, Flink, Trino, Snowflake
Apache Hudi	Copy-on-Write or Merge-on-Read	Upserts; incremental queries	Streaming ingestion, CDC-style
Delta Lake	Transaction log over Parquet	ACID, DML, versioning	Databricks, Spark

Streaming to the Lakehouse

- Kafka → Spark Structured Streaming → Delta Lake: standard streaming ingestion pattern
- Kafka → Flink → Iceberg: alternate stack with lower latency
- Autoloader (Databricks): incrementally ingests new files from object storage into Delta
- Schema evolution: detect schema changes in incoming streams and evolve tables automatically
- Watermarks: handle late-arriving events; decide how long to wait before closing a window

Resources & Further Reading

- Books:
  - The Data Warehouse Toolkit — Ralph Kimball & Margy Ross (3rd ed.)
- Docs and whitepapers:
  - Databricks Lakehouse whitepaper: databricks.com/product/data-lakehouse
  - Apache Iceberg docs: iceberg.apache.org/docs
  - Apache Hudi docs: hudi.apache.org/docs
  - Delta Lake docs: docs.delta.io

Summary

- Cloud warehouses separate storage and compute for petabyte-scale analytics.
- Dimensional modeling with facts and dimensions powers BI reporting.
- Lakehouses add ACID transactions to cheap object storage.
- Bronze, Silver, and Gold layers progressively refine data quality.
- Design governance — catalog, lineage, masking — in from the start.



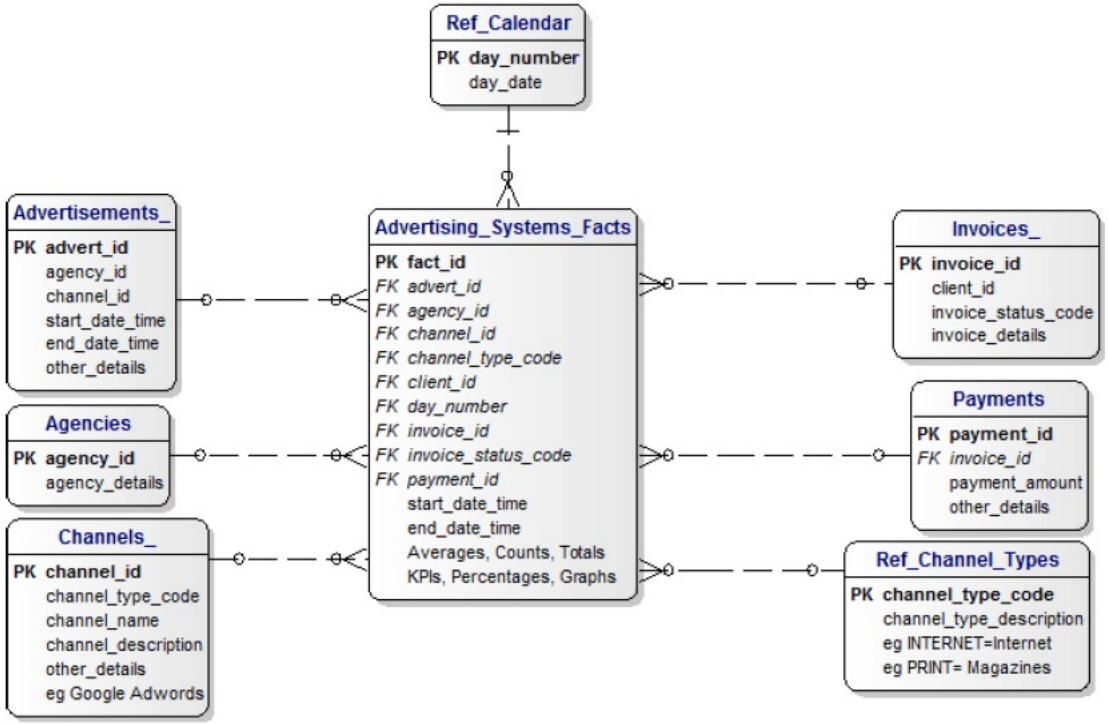
Appendix

Appendix

Case Study 2 : Ads

- Advertisements are placed by Agencies.
- Advertisements are placed by Channels, such as TV or Print.
- Advertisements can be part of a Marketing Campaign(Targeting).
- Invoices are produced and result in Payments.
- Advertisements result in Responses.
- Responses can be analyzed by Demographics. (Measurement)

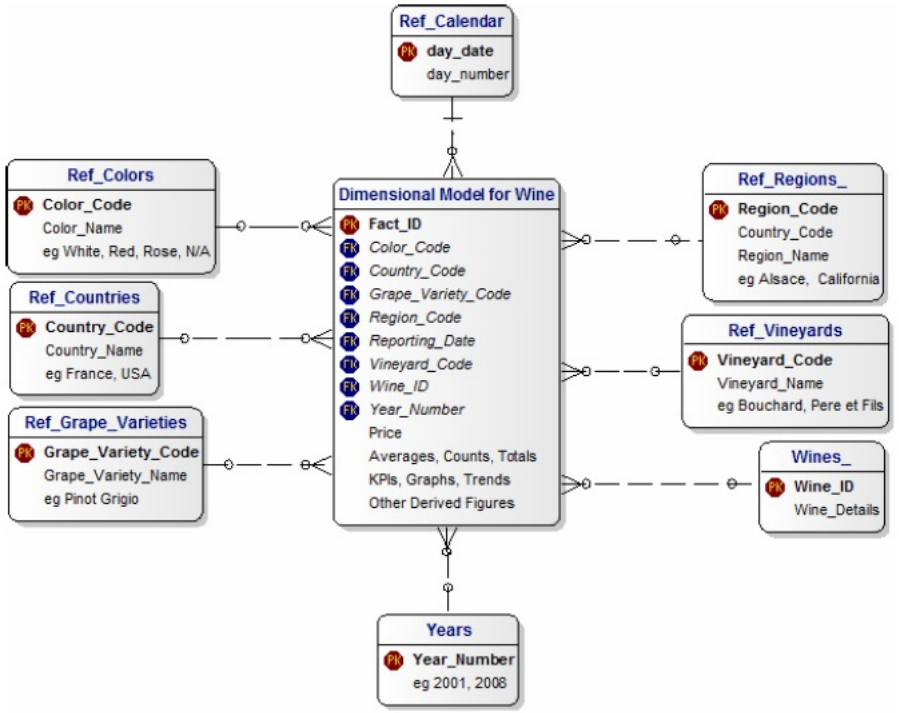
Dimensional Model



Case Study 3 : Wines

- Wines are defined by a number of characteristics, including Grape Variety, Year, Vineyard, Color, Country and Region.
  - A Region is always in a country
  - A wine is always associated with a country
  - A wine is sometimes associated with a region
  - Every wine has color, including champagne
  - Every wine has a year
  - Non-Vintage Champagnes do not have a year. Therefore the relationship between the years and wine is optional at the years end.

Dimensional Model



Case Study 4 : Audience

- Customer demographics - Countries, Customer Categories and Gender.
- Customers place Orders.
- Orders involve Shipments that deliver goods to Customers.
- Shopping Carts are used to hold Customer’s goods.
- Mailshot Campaigns target specific Mailshot Customers
- Customers choose Customer Payment Methods from a standard list of Payment Methods.

Dimensional Model

